

OPTIMIZING VALUE AT RISK CALCULATION USING SUPPORT VECTOR MACHINES

by
Naveed Ahmad, B.A.

University College London
2006

Presented in Partial Fulfillment of the Requirements for the Degree
MSc Intelligent Systems in the Department of Computer Science at
University College London. This study is substantially the result of
the work of the author except where explicitly indicated in the text.

The report will be distributed to the internal and external
examiners, but thereafter may not be copied or distributed
except with permission from the author.

Advised by:

Prof. Philip Treleaven

ABSTRACT

There is need for Intelligent Systems implementations catered to the demands of risk managers in the financial risk industry. A wealth of research studies exist which employ Intelligent Systems for approximating Value-at-Risk (VaR), a financial market risk measure. Working with the Enterprise Risk System's (ERS) team at Calypso Technology it became evident that the risk managers in the industry are uninterested in these approximation algorithms due to their inability for producing accurate VaR approximations for non-linear financial instruments. The risk managers require instrument independent optimization of the robust non-parametric approach, Historical Simulation(HS), which calculates VaR by revaluing trades on historical market scenarios. The aim of this dissertation was explore different Intelligent Systems in the wake of the requirements of the risk industry. Four experiments were carried out to explore the feasibility of different Intelligent Systems for HS optimization:

Searching Historical Trading scenarios using Genetic Algorithms: Theoretical framework using Genetic Algorithms (GA) was designed to search the historical market scenarios. The motivation was to harness the GA's capability in avoiding local minimums, an attribute of their population feature. The idea of their implementation was abandoned in light of their infeasibility as a practical solution owing to the lack of an appropriate stopping criteria.

Approximating pricing model using Support Vector Regression: In this experiment Support Vector Machines were used to approximate the exogenous pricing model. Approximation was employed in light of a black box computationally expensive pricing model, where a decent approximation from the SVM would identify which market days to re-price via the pricing model

to calculate the VaR measure. This methodology although unsuccessful at approximating the non-linear pricing model gave insight into problem domain.

Support Vector Classification(SVC) on P&L: Learning from previous experiments the SVC was designed to classify historical data into two classes. The ideology was restructured by making the objective of SVM training from approximating the pricing model to classification of historical market scenarios in terms of profit and loss. This methodology on average reduced the computational time of Historical Simulation by 50%.

Support Vector Regression(SVR) on P&L: In effort to further optimize HS SVR was employed on a simple hard limit profit/loss vector. This methodology surpassed the SVC in computational time and accuracy. A further increase in the prediction accuracy in SVR was gained by using differential genetic algorithms to search for optimal kernel parameters. This novel technique reduces the computational time of the HS by 74%, while producing accurate VaR measures for non-linear portfolios.

The principle accomplishment of this dissertation are that it successfully met the aforementioned criteria by: i) Implementing a model independent Intelligent System that can be used for any portfolio of non-linear instruments. ii) Combined simple classification with support vector regression to optimize the Historical Simulation by achieving accurate results in one fifth of the time. This implementation is also the first known practicable application of Intelligent Systems in optimization of Historical Simulation methodology of calculating VaR. The study's exploration of different algorithms presents new knowledge about the limitation of Genetic algorithms and novel implementation techniques of Support Vector Regression. This study also recommends to modify Regression by feature projection as further research in the optimization of HS.

ACKNOWLEDGMENTS

I would like to acknowledge

Amir Khawaja, Director of Risk Management at Calypso Technology, for proposing the problem, and providing the necessary support and guidance.

Prof. Philip Treleaven, Pro-provost Professor University College London, for all the inspiration, insight, and help. I am really going to miss our long conversations.

Risk Management Team at Calypso Technology, UK, for all the support, help, and suggestions.

My loving parents for always being there for me.

CONTENTS

Abstract	i
Acknowledgments	iii
CHAPTER	PAGE
1 Introduction	1
2 Background of the Problem: Quantifying Market Risk, Value at Risk	3
2.1 Value-at-Risk Measure	4
2.2 Overview of Risk Methodologies	5
2.3 Covariance VaR Model	7
2.4 Simulation VaR Models	9
2.4.1 Historical Simulation	9
2.4.2 Monte Carlo VaR	11
2.4.3 Summary of Risk Models	11
3 Methodology of Intelligent Systems Proposed	14
3.1 Support Vector Machines	14
3.1.1 Data Representation and Similarity Measures	15
3.1.2 A basic overview of Statistical Learning Theory	16
3.1.3 Optimal Separating Hyperplane	21
3.1.4 Support Vector Classification	24
3.1.5 Support Vector Regression	25
3.2 Genetic Algorithms	27
4 The System Developed: SVM Historical Simulation Optimizer	32
4.0.1 The non-linear pricing model: Bermudan Swaption	36
4.0.1.1 Example of a Swaption: Case European Swaption	36
4.0.2 Swaption Pricing Model: Hull White Single Factor Model	37
5 Experiment: Intelligent System Optimization of Historical Simulation	39
5.1 Genetic Algorithms	40
5.2 Support Vector Regression on Prices	42
5.3 Support Vector Classification on P&L using simple classification	43
5.3.1 Case: Optimizing Historical Simulation using 2-Class classification	44
5.3.2 Case: Optimizing Historical Simulation using 3-Class classification	45
5.4 Support Vector Regression on P&L and using simple Classification	46

5.4.1	Case: Optimizing Historical Simulation using 2-Class classification	46
5.4.2	Case: Optimizing Historical Simulation using 3-Class classification	47
5.4.3	Case: Optimizing Historical Simulation using P&L	48
5.5	Analysis of SVM Experiments	49
6	Conclusion	53
	References	55

INTRODUCTION

Within the last 10 years, the financial world has seen the breakdown of several large institutions due to large exposures to market specific movements. In the wake of these financial disasters it has become increasingly clear to the world of finance that better means of controlling market induced financial risk is necessary. This has given rise to the development of good market risk measures of which the most commonly used is Value-at-Risk (VaR).

The elegance of VaR solution is that it works on multiple levels from the position specific micro level to the portfolio based macro level. VaR has become a common language of analysts, regulators, rating agencies, and shareholders (Laubsh, 1999). This measure can be calculated by many different methods both parametric and non-parameter. Only three main methods are accepted by the regulator, Basel Committee, and the banking industry: Historical Simulation, Covariance Methods, and Monte Carlo Simulation.

During the last few years the practitioner's preferred choice has been the use of non-parametric techniques, mainly Historical simulation(HS). In HS a number of scenarios are created for each risk factor of all the positions(trades) using historical market data. These positions are priced using a full pricing model and then aggregated to form the set of scenarios for portfolio value. VaR is then calculated on the Profit and Loss distribution of these scenarios.

Accurate VaR measures through Historical Simulation requires at least two years worth of historical market scenarios. In case of large portfolios holding non-linear derivatives Historical Simulation becomes computationally intense. Pricing these trades over a minimum of two years of market scenarios requires hours of CPU time. Many techniques have been devised to approximate

that Value-at-Risk measure, but none focus on optimizing the currently used Historical Simulation method.

At Calypso Technology, London, the Enterprize Risk Management(ERS) team is faced with increasing concerns from prospective clients about the amount of time it takes their risk management software to calculate VaR using Historical Simulation. Conventional optimization can be achieved through better programming of the software tool, but since the software is developed in Java few options are available. The purpose of this study was to devise an instrument independent optimization of Historical Simulation for Calypso Technology through experimentation with different Intelligent System techniques.

BACKGROUND OF THE PROBLEM: QUANTIFYING MARKET RISK, VALUE AT RISK

Businesses manage risk on a daily basis to minimize potential loss in order to ensure the viability, profitability, and good reputation of their respective institutions (Harmantzis, Miao and Chien, 2005). One of the types of risks managed by the businesses is called financial risk. Financial risk is any type of risk that involves a monetary loss due to changes in the financial markets. There are many types of financial risk of which the main are: operational risk, business risk, credit risk, and market risk. In this dissertation only market risk is considered, which can be easily quantified by one measure, Value-at-Risk.

The growth of the over-the-counter(OTC) derivatives market has created a need to measure and manage the risk of portfolios whose value fluctuates in a nonlinear way with changes in the risk factors. “One the most widely used risk measure is the Value at risk measure or VaR” (Laubsh, 1999). A portfolios VaR is the most that the portfolio is likely to lose over a given time horizon except in a small percentage of circumstances (Engelbrecht, 2003). This expected loss is commonly referred to as the VaR confidence level. For example, if a portfolio is expected to lose no more than 10 million over the next day except in 1 in 100 days then its VaR is one million at 99% confidence level. Alternately, the portfolios VaR at $k\%$ confidence level is the $k'th$ percentile of the distribution of the change in the portfolios value of the VaR time horizon (Jorion, 2002).

The main advantage of VaR as a risk measure is that it is very simple to understand. It can be used to summarize the risk of individual position or of large multinational financial institutions,

such as the large dealer-banks in the OTC derivatives markets. Due to VaRs simplicity, it has been adopted for regulatory purposes. More specifically, the 1996 Market Risk Amendment to the Basle Accord stipulates that banks and broker-dealers minimum capital requirements for market risk should be set based on the ten-day horizon 1-percent VaR of their trading portfolios.

2.1 VALUE-AT-RISK MEASURE

“VaR is defined as the loss (stated with a specified probability) from adverse market movements over a fixed time horizon, assuming the portfolio is not managed during this time” (Jorion, 2002). VaR is measured as a lower percentile of a distribution for theoretical profit and loss that arises from the possible movement of the market risk factors over a fixed risk horizon. VaR is formally stated as

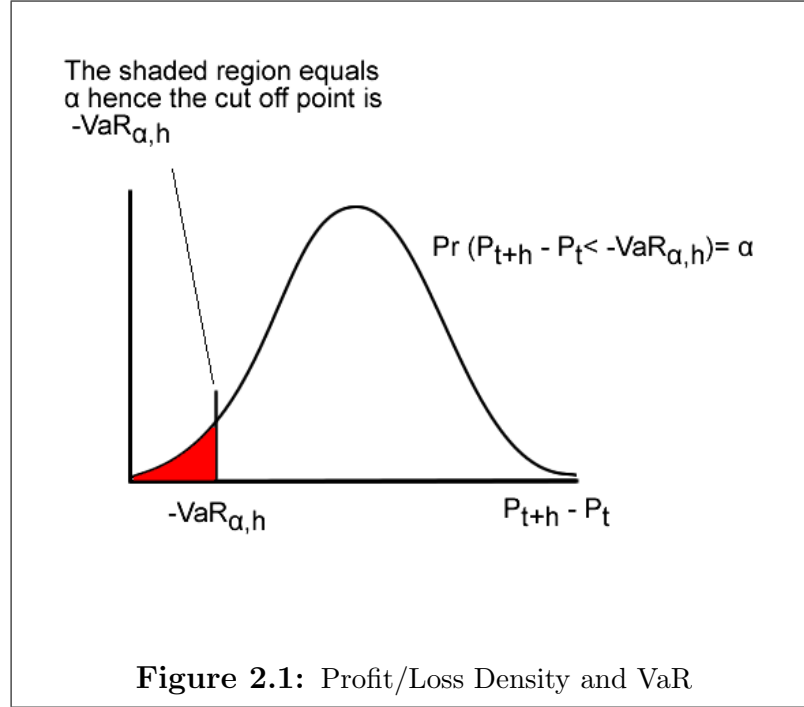
$$\Delta_h P_t = P_{t+h} - P_t \quad (2.1)$$

The $\Delta_h P_t$ is the future h day theoretical Profit/Loss(P&L) obtained by simply marking the portfolio to market today and then leaving it unchanged and marking it to market again at the risk horizon.

The probability that is associated with a VaR measurement is called the confidence level of VaR. This corresponds to the frequency with which a given level of loss is expected to occur. So for example a 95% one day risk horizon VaR corresponds to a loss level that one expects to exceed, in normal market circumstance, one day in twenty. A 99% one day horizon VaR is the level of loss that might be seen one day in a hundred. The definition of VaR above can be rephrased as follows: the $100\alpha\%$ h -day VaR is that number x such that the probability of loosing x , or more, over the next h days equals $100\alpha\%$. Since VaR is dependent on two parameters α , the confidence level, and h , the holding period hence mathematically it can be written as,

$$Pr(\Delta_h P_t < -VaR_{\alpha,h}) = \alpha \quad (2.2)$$

Equation 2.2 shows that the $100\alpha\%$ h-day VaR measurement is the lower quantile of the unrealized profit/loss distribution. Figure 2.1 summarizes this:



VaR estimates increase with an increase in the confidence level chosen and the holding period. The VaR increases as the holding period increases because the volatility will generally increase with holding period. To calculate VaR in horizon of ten days, the square root of time rule is employed so that the h-day VaR is simply taken as $\sqrt{h}$ time the 1-day VaR. If this rule is not invoked then the VaR measure is simply the lower percentile of the historical h-day P&L distribution or, when a covariance matrix is used, it is the VaR based on the h-day covariance matrix.

2.2 OVERVIEW OF RISK METHODOLOGIES

Computing VaR in practice can be very difficult even though conceptually VaR is a simple measure. VaR depends on the joint distribution of all of the instruments in the portfolio. For large financial firms which have tens of thousands of instruments in the portfolio, simplifying steps are usually employed as part of the VaR computation. Three steps are generally used. 1) The dimension of the problem is reduced by modeling the change in the value of the instruments in the portfolio

as depending on a smaller (but still large set of risk factors f . 2) The relationship between f and the value of the instruments which are non-linear functions of f is approximated where necessary. Finally, an assumption about the distribution of f is required (Holton, 2004).

The errors in VaR estimation depend on the reasonableness of the simplifying assumptions. One of the most important assumptions is the underlying probability distribution of the risk factors. Many large banks currently use a method known as historical simulation to calculate VaR. “The distinguishing feature of the historical simulation methods and its variants is that they make minimal parametric assumptions about the distribution of f , beyond assuming that the distribution of changes in value of today’s portfolio can be simulated by making draws from the historical time series of past changes in f ” (Holton, 2004).

Three main methods accepted by the regulator, Basel Committee, and the banking industry are Covariance, Historical Simulation, and Monte Carlo Simulation methods. These can be classified into two main classes of methodologies: parametric and non-parametric. The Covariance method is a parametric approach, while the two simulations, Historical and Monte Carlo, are non-parametric approaches. Each of these methods has its strengths and weaknesses, but combined they give a more comprehensive perspective of market risk. To get a brief summary of each method class please refer to Table 2.1.

Methodology	Description	Applications
<i>Covariance</i>	Estimates VaR with equation that specifies parameters such as volatility, correlation, delta, and gamma	Accurate for traditional assets and linear derivatives, but less accurate for non-linear derivatives
<i>Monte Carlo Simulation</i>	Estimates VaR by simulating random scenarios and revaluing positions in the portfolio	Appropriate for all types of instruments linear and non-linear
<i>Historical Simulation</i>	Estimates VaR by reliving history. It takes actual historical scenarios and revalues positions for each change in the market	Appropriate for all types of instruments linear and non-linear

Table 2.1: Three main methodologies of VaR (Laubsh, 1999)

Before allowing for a formal description of the methodologies, it is important to understand the difference between linear and non-linear financial instruments. Linear financial instruments have price changes which are proportionately relative to the movements of the underlying assets, while the non-linear instruments have price changes which are disproportionate. Traditional linear instruments include stocks, bonds, and futures, while examples of non-linear instruments are options and swaptions. To calculate disproportionate payoffs or prices of non-linear instruments like swaptions it is recommended that full valuation formulas be implemented (Laubsh, 1999).

In the following sections a mathematical description of the covariance and the historical simulation methods will be given. The mathematical framework through which the Monte-Carlo simulation generates hypothetical market scenarios is out of the scope of this dissertation and therefore will not be included. **In conclusion a comparison of these methodologies is made with a brief summary on research in other simulation methodologies.**

2.3 COVARIANCE VAR MODEL

The covariance VaR method was developed by JP Morgan and introduced to the financial community on October 1994. In this VaR methodology the only data necessary to compute the VaR of a linear portfolio is a covariance matrix of all the assets in the portfolio. It requires the knowledge of the portfolio composition, and the variances and covariances of the asset returns. These can be calculated using the GARCH methodology (Holton, 2003). The Basel Committee recommends at least one years historic data be used to calculate the variance and covariances (Jorion, 2002).

To explain the covariance method in the mathematical terms, we first assume that the Profit and Loss (P&L) positions of the returns are normally distributed. The covariance VaR Formula is then computed as follows:

If $\Delta_h P_t = P_{t+h} - P_t$ denotes the h-day unrealized P&L, we assume

$$\Delta_h P_t \sim N(\mu_t, \sigma_t^2) \quad (2.3)$$

The $100\alpha\%$ h -day VaR is that number $VaR_{\alpha,h}$ such that,

$$Pr\left(\frac{\Delta_h P_t - \mu_t}{\sigma_t} < \frac{-VaR_{\alpha,h} - \mu_t}{\sigma_t}\right) = \alpha \quad (2.4)$$

or since $\frac{\Delta_h P_t - \mu_t}{\sigma_t} \sim N(0, 1)$, and denoting $\frac{\Delta_h P_t - \mu_t}{\sigma_t}$ by the standard normal variant Z_t ,

$$Pr\left(\frac{\Delta_h P_t - \mu_t}{\sigma_t} < \frac{-VaR_{\alpha,h} - \mu_t}{\sigma_t}\right) = \alpha \quad (2.5)$$

Now standard normal variant Z_t ,

$$Pr(Z_t < Z_\alpha) = \alpha \quad (2.6)$$

where Z_α is the 100α th percentile of the standard normal density. Therefore,

$$\frac{-VaR_{\alpha,h} - \mu_t}{\sigma_t} = -Z_\alpha \quad (2.7)$$

re-written, we get the covariance VaR formula:

$$VaR_{\alpha,h} = Z_\alpha \sigma_t - \mu_t \quad (2.8)$$

VaR as a risk measure is only suitable for short-term risk, so it is normal to assume that the $\mu_t = 0$. Now, Z_α is simply a constant given in the standard normal tables¹ Thus it is the volatility of the P&L, σ_t , determines the portfolio VaR.

The covariance method has a couple of advantages. It is quick and simple to compute, hence is great for intra-day VaR calculations. These fast but rough approximations enables traders to find the impact of a proposed trade on their VaR limit. It also requires less market scenarios to generate the results. The use of more recent market scenarios allows for more current market conditions to be used in the calculations of the VaR measure, therefore having a similar volatility to the risk horizon. However the disadvantage of this approach is that it has very limited applicability and is

¹Ex: For the 95% confidence level, the $\alpha = 0.025$ and the $Z_\alpha = 1.96$ in the standard normal table.

only reliable for linear portfolios or portfolios that are assumed to be linear with respect to the risk factors.

2.4 SIMULATION VAR MODELS

In the past few years the use of simulation methods of VaR analysis has become a standard due to the development of the Over-The-Counter(OTC) derivatives market. The OTC derivatives market comprises of non-linear instruments. The simulation methods are more accurate in calculating VaR of non-linear instruments, therefore they are preferred over the parametric methods such as the covariance method. Two simulation methods are most popular in the risk industry. The Historical Simulation and the Monte-Carlo Simulation. The Historical Simulation method and the Monte Carlo methods are mechanically identical since they both revalue instrument to changes in the market rates. The difference between the two methodologies lie in their method of generating market scenarios. The historical simulation uses actual past market movements as scenarios while the Monte-Carlo simulates random hypothetical scenarios.

2.4.1 HISTORICAL SIMULATION

The historical simulation uses real historical market scenarios to build an empirical profit/loss density for an instrument or a portfolio of instruments. No assumptions about the analytic form of this distribution is made, and neither about any co-movement between assets(comprising of instruments) or the risk factors. This method is also able to accurately calculate VaR for non-linear assets. It also less computationally intense than the Monte-Carlo simulation method. Hence, Historical Simulation in particular is an extremely popular choice for many institutions (Jorion, 2002).

In Historical Simulation methodology, scenarios are collected on the underlying assets on a daily basis covering many years. The Basel Committee recommends at least one year's worth of market scenarios for accurate results, but it is preferred that at least 2 to 3 years is used (Holton,

2004). using these scenarios the portfolio is re-valued for each of the historic scenarios, keeping the composition of assets in the portfolio constant.

For a formal explanation, let us consider a linear portfolio with a h -day portfolio return $\frac{\Delta_h P_t}{P_t}$ is the weighted sum of the return R_i to risk factors calculated as:

$$\frac{\Delta_h P_t}{P_t} = w_i R_{i,t} + \dots + w_k R_{k,t} \quad (2.9)$$

Historical data is gathered on each R_i , and the portfolio price changes over h days are simulated as follows:

$$\Delta_h P_t = \sum (w_i P_t) T_{i,t} = \sum p_i R_{i,t} \quad (2.10)$$

Where the p_i are the actual amount invested in each asset. This representation allows h -day theoretical P&Ls for the portfolio to be simulated from the historical data $R_1 \dots R_k$ ².

The empirical h day P&L is obtained through building a histogram of the difference between the current market day and the historical days calculated as such:

$$\Delta_h P_t = P_{t+h} - P_t \quad (2.11)$$

The historical $VaR_{\alpha,h}$ is the lower $100\alpha^{th}$ percentile of this distribution. This VaR will be sensitive to the historical period chosen to evaluate the portfolio. If for example if 500 days were chosen to create the empirical density, the lower 1% tail will be made up of 5 largest losses in the period. The VaR will be the smallest of these five largest losses.

For robustness of the estimate it is apparent that longer periods of historical data must be chosen, introducing a number of concerns. The historical period chosen might not be a good representative of the future period for which the VaR is to be calculated. For example, if the historical scenarios include periods of high volatility, the market crash of the US markets in 2001, and such a situation is not expected to take place in the risk horizon, then the VaR estimates produced by the HS method

²Usually a full evaluation is carried out for the entire portfolio. Sometimes approximations are also used for standard products like options, swaps, and swaptions.

will be inaccurate. Therefore it is important that the historical market scenarios chosen reflect the expected volatility in the h -day horizon.

2.4.2 MONTE CARLO VAR

Monte Carlo simulation generates movements in underlying assets and risk factors from the current market day to some future point in time, h . It takes the values of the current market day as the starting point and generates thousand of values of risk factors of the underlying assets for the risk horizon. This large set of scenarios is then used to obtain thousand of possible values for the portfolio in h days time. A histogram of the difference between these and the current portfolio value is obtained. Similar to the historic simulation method, the VaR measure is simply the lower percentile of this distribution.

It is necessary that the data generated mimics the actual real life situations. For example, a two year swap rate increases by twenty basis points at the same time as the three year swap rate decreases by fifty basis points is not reasonable. Not only will the volatility of an asset determine its possible future values in h -days time, but also the correlations between different assets in the portfolio also need to be catered for. Hence usually an h day covariance matrix is employed for all the underlying assets and risk factors in the portfolio (Holton, 2004).

The main advantage of Monte Carlo simulation is that it is widely applicable, and it is able to capture path dependent behavior of complex products (Holton, 2004). The main disadvantage of MC VaR is that it needs a covariance matrix to compute results. The use of covariance matrices introduces another well known source of error. Often accuracy will need to be compromised in order to complete the VaR calculations within a realistic time frame (Jorion, 2002).

2.4.3 SUMMARY OF RISK MODELS

The VaR models discussed in this section are the most commonly used models in the financial industry. They are implemented in various scenarios with respect to their accuracy and computational performance. Each of these models have their advantages and disadvantages, a summary is given in Table 2.2 .

Methodology	Advantage	Disadvantage
<i>Parametric</i>	▷ Fast and simple calculations ▷ No need for extensive historical scenarios only volatility and correlation matrix is required	▷ Less accurate for nonlinear portfolios, or for skewed distributions
<i>Monte Carlo Simulation</i>	▷ Accurate* for all instruments ▷ Provides a full distribution of potential portfolio values not just a specific percentile ▷ Permits use of various distributional assumptions(normal, T-distribution, etc.) and therefore has potential to address the issue of leptokurtosis	▷ Computationally intensive and time-consuming. It involves revaluing the portfolio under each scenario ▷ Quantifies fat-tailed risk only if market scenarios are generated from the appropriate distribution
Historical Simulation	▷ Accurate* for all instruments ▷ Provides a full distribution of potential portfolio values not just a specific percentile ▷ No need to make distributional assumptions. ▷ Faster than Monte Carlo simulation because less scenarios are used	▷ Requires a significant amount of daily history. However, sampling far back in history may make use of data that is irrelevant to current market conditions ▷ Difficult to scale far into the future. ▷ Coarse at high confidence levels (99% and beyond) ▷ Somewhat computationally intensive and time-consuming. Involves re-valuing although the portfolio under each scenario, far less scenarios are required than for Monte Carlo ▷ Incorporates tail risk only if historical data set includes tail events.

*Accurate if used with complete pricing algorithm.

Table 2.2: Strength and Weakness of the three main models (Laubsh, 1999)

Most financial concepts which have been developed in the past decade assume that the returns of portfolios follow a normal distribution. However empirical results show that extreme events such as losses or gains, occur with a higher probability than implied by the assumption of the normal distribution (Harmantzis, Miao and Chien, 2005). This causes the P&L distribution to suffer from leptokurtosis(fat tails). Hence nullifying the previous assumption. Due to the leptokurtic nature of the returns of non-linear financial instruments most parametric techniques perform badly in

calculating value at risk. Thus especially for non-linear portfolios simulation techniques such as Historical and Monte Carlo are used.

Monte Carlo Simulation is extremely time intensive unless methods are used to speed up its calculations. A recent study applied Delta-Gamma approximation using Cornish Fisher Expansions and Johnson Transformations to speed up the Monte Carlo Simulation (Mina and Ulmer, 1999). The results are unimpressive. Therefore, the most logical choice of calculating VaR for complex non-linear instruments in the real world is HS.

No significant research has been performed on optimizing computational intensity of Historical Simulation methodology. Most studies concentrate on improving Monte Carlo Simulation. Considering that a majority of the bank and financial institutions in the world use Historical Simulation it would provide Calypso Technology Enterprize Risk Software an edge in the market if it could provided an optimized Historical Simulation.

METHODOLOGY OF INTELLIGENT SYSTEMS PROPOSED

This section will introduce the fundamentals of support vector machines and genetic algorithms, the two main Intelligent Systems considered for optimization of the Historical Simulation. The idea of support vectors was initially introduced in 1996 by Vapnik¹. A basic introduction will be given in this section which has been adopted from Learning with Kernels by Scholkopf and Smola, 2002. Most of the arguments have not been substantiated as sound reasoning of Statistical Learning Theory is beyond the scope of this dissertation.

3.1 SUPPORT VECTOR MACHINES

Support Vector Machines were chosen because of many attractive features, and promising empirical performance. There are two classes of Support Vector Machines: Support Vector Classifier and Support Vector Regression. The major difference between SVMs and their conventional counterparts neural networks is that the former uses structural risk minimization for learning while the later employs the traditional empirical risk minimization techniques. A basic review of the statistical methodology behind the success of support vector machines is necessary to get a whole sum idea of their generalization abilities.

Support Vector Machines belong to a class of Structural Risk Minimizers.

¹For a more in-depth understanding of Support Vector Machines refer to “Statistical Learning Theory” by Vladimir N. Vapnik, 1996

3.1.1 DATA REPRESENTATION AND SIMILARITY MEASURES

An Elementary example in Statistical Learning Theory:

Suppose we are given a set of empirical data formally denoted as:

$$(x_1, y_1), \dots, (x_m, y_m) \in X \times \pm 1 \quad (3.1)$$

This data contains a non empty set of patterns x_i (inputs or observations), which are mapped to y_i (labels or targets). In our case the targets are binary in nature as they contain values $+1$ and -1 . The patterns x could be anything hence no assumptions are made at this point.

The task in term of learning theory, is to classify new unseen observations $x \in X$ to the given labels $y \in \pm 1$. This can be achieved simply by matching the new inputs to the empirical data and characterizing them based on a similarity measure to the outputs. Formally this similarity measure is defined as

$$k : X \times X \rightarrow \mathbb{R}$$

$$(x, x') \mapsto k(x, x') \quad (3.2)$$

The similarity measure defined above will return a real number quantifying the similarity between the patterns x and x' . For simplicity let us assume that the above similarity measure is a *dot product*. Hence for the two vectors $x, x' \in \mathbb{R}^N$, the *canonical dot product* is defined as:

$$\langle \mathbf{x}, \mathbf{x}' \rangle := \sum_{i=1}^N [\mathbf{x}]_i [\mathbf{x}']_i \quad (3.3)$$

The canonical dot product computes the cosine between the vectors x and x' . This allows for the computation of the length (or norm) of the vector x .

$$\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle} \quad (3.4)$$

The difference of the two vectors is computed as the length of the difference vector given above as $\|\mathbf{x}\|$.

Unfortunately, the dot product approach is not sufficient in general to deal with real world problems² since it cannot be claimed that the pattern actually exists in a dot product space. This pattern could be any kind of object. In order to use this dot product as a similarity measure, the pattern needs to be represented as a vector in some dot product space $\mathcal{H}$. This can be performed mathematically using a map as follows:

$$\Phi : X \rightarrow \mathcal{H}$$

$$x \mapsto \Phi(x) \quad (3.5)$$

The space $\mathcal{H}$ is called the *feature space*. By defining the data in a feature space $\mathcal{H}$ via the map Φ we gain three benefits:

1. A similarity measure from the dot product in feature space $\mathcal{H}$, which can be defined as follows:

$$k(x, x') := \langle x, x' \rangle = \langle \Phi(x), \Phi(x') \rangle \quad (3.6)$$

2. A pattern which can now be dealt with geometrically, and therefore can be studied using learning algorithms using linear algebra and geometry.
3. The ability to use of any kind of mapping Φ which can be used to design a large variety of similarity measures and learning algorithms. For example, a non linear Φ can be used to map the original vector and change its current representation into one that is more suitable for better generalized learning.

3.1.2 A BASIC OVERVIEW OF STATISTICAL LEARNING THEORY

In this section a few basic argument of statistical learning theory will be covered which will enable for a more sound basis to introduce Support Vector machines.

In binary pattern recognition the objective is to approximate a function,

²Later in this chapter it will be shown why that is the case.

$$f := x \rightarrow \pm 1 \quad (3.7)$$

from training data consisting of inputs and outputs. This training data is often referred to as the sample data.

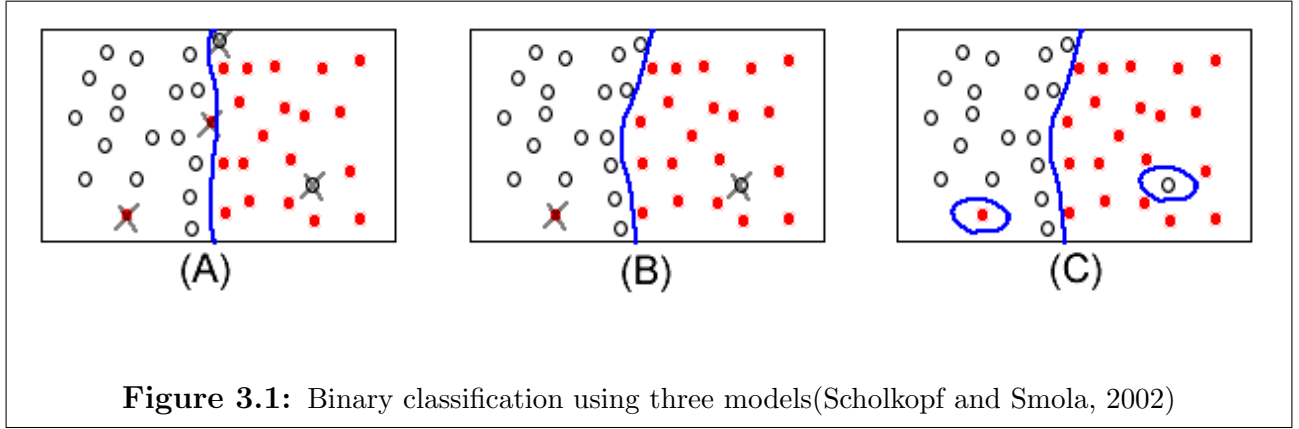


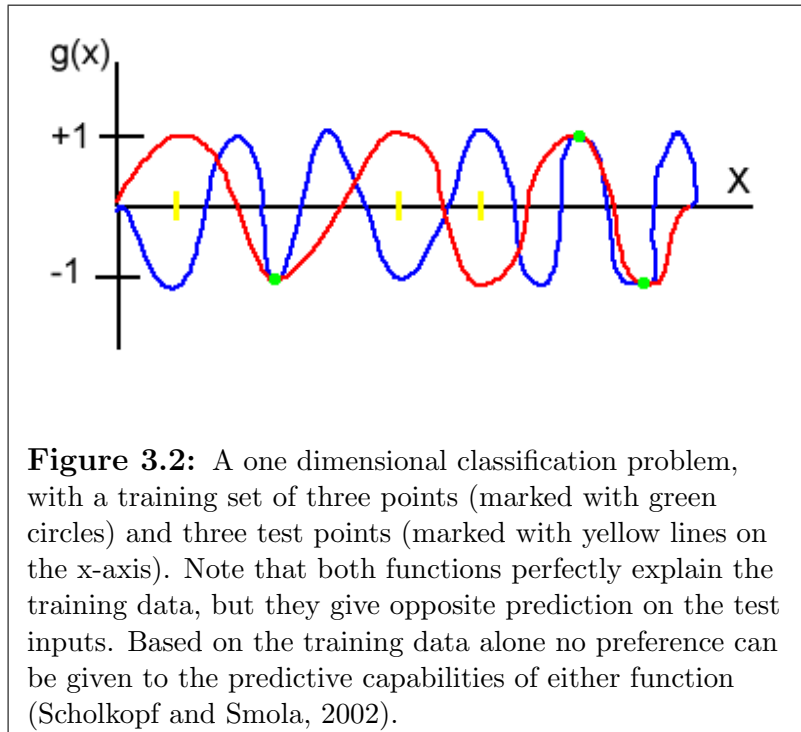
Figure 3.1 shows an illustration of the training data of a pattern recognition problem where three different models of the function f have separated the data into two classes. The hollow black dots in the illustration have a value of -1 while the solid red dots have a value of $+1$. Instead of drawing the model functions only the boundaries where the function switches between -1 to $+1$ are shown. Looking at model (c) it can be seen that the function correctly separates the red from the black dots. It even correctly classifies the outliers: the black dot in the red majority area and the red dot in the black majority area. However it is questionable that when the test data, unseen observations, are presented from a similar underlying distribution whether the function will label them correctly. For example, if a test data point lies close to the red dot outlier, then the function might label it as $+1$. Hence it is logical to go for a simpler function that is not marred by this issue. Such a function is present in the model labeled (a). The function uses an almost linear separator to divide the two classes. Unfortunately this function does a poor job of approximating the sample data. It misclassifies point close to classification divider which could easily have been correctly labeled. Hence the optimal classifier is produced by the function shown in the model labeled (b) in Figure 3.1. This correctly classifies the points near to the classification divider. The misclassification of the outliers is acceptable in light of the better classification of the test data.

Statistical learning theory puts the intuitive arguments such as the ones made about Figure 3.1 into a mathematical framework. It can therefore be claimed that the statistical learning is a study of the mathematical properties of learning machines. “These properties are usually the properties of the function classes that the learning machine can implement” (Scholkopf and Smola, 2002).

Our original goal was to find a function f that will learn from training data to correctly classify unseen examples, testing data. Lets assume that the training and testing examples are coming from an unknown fixed probability distribution denoted as, $P(x, y)$. Data belonging to such a probability distribution is commonly referred to as independent and identically distributed (*iid*). Correctness of the function is measured by a zero-one loss function,

$$c(x, y, f(x)) := \frac{1}{2}|f(x) - y| \quad (3.8)$$

This loss function is zero if x is classified correctly and 1 otherwise. There is no restriction on the set from which our function is chosen. Consequently the choice function f may perform really well on the sample data, but no guarantees can be made on its generalization capability on the test examples. Figure 3.2 illustrates this situation with a one dimensional classification problem.



The situation in Figure 3.2 can formally be summarized as:

For each function f , a training set

$$(x_1, y_1), \dots, (x_m, y_m) \in X \times \pm 1 \quad (3.9)$$

, and a test set

$$(a_1, b_1), \dots, (a_m, b_m) \in X \times \pm 1 \quad (3.10)$$

both drawn from the same underlying distribution $P(x, y)$, therefore $a_1, \dots, a_m \cap x_1, \dots, x_m = \emptyset$. Now there exists a function $\hat{f}$ such that $\hat{f}(x_i) = f(x_i)$ for all $i = 1, \dots, m$. Yet $\hat{f}(a_i) \neq f(a_i)$ for all $i = 1, \dots, m$.

Hence we can conclude that minimizing only the average training error (or empirical error),

$$R_{emp}[f] = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} |f(x_i) - y_i| \quad (3.11)$$

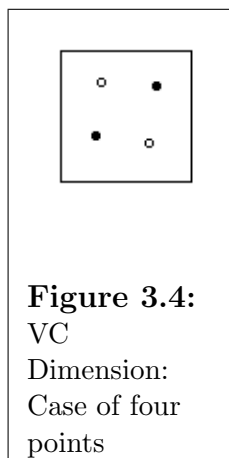
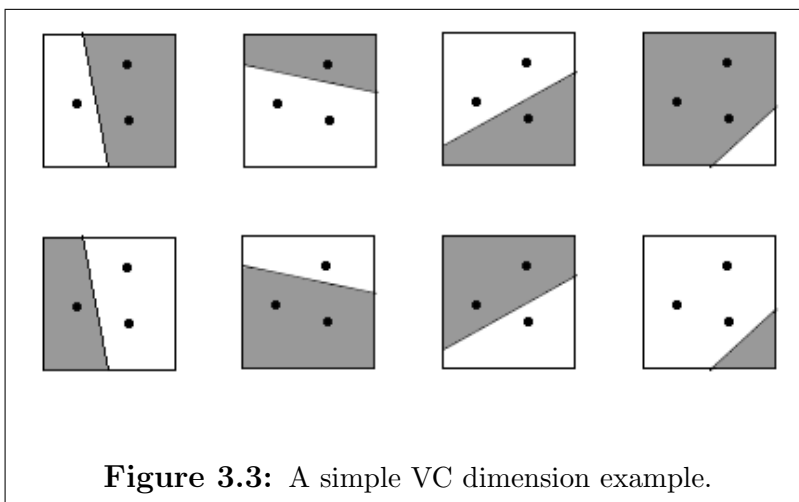
does not imply a small test error (risk), averaged over the test examples drawn from the underlying distribution $P(x, y)$,

$$R[f] = \int \frac{1}{2} |f(x) - y| dP(x, y) \quad (3.12)$$

The above conjecture can be made for any loss function provided the integral exists. For the current zero-one loss function, the risk equals the probability of misclassification. The ultimate goal of is to find a function f such that is minimizes risk (test error), which is carried out by using the principal of structural risk minimization in Vapnik Chervonenkis (VC) theory.

The VC theory in Statistical Learning, shows that it is imperative to restrict the set of functions from which f is chosen to one that has the capacity suitable for the amount of available training data. This theory provides the bounds on the test error. The minimization of these bounds depends on both the empirical risk and the capacity of the function class. This leads to the principle of structural risk minimization.

The best-known capacity concept of the VC theory is the VC dimension. The VC dimension is a scalar value that measures the capacity of a set of functions. The capacity of a function class $f(x, \alpha)$, is defined to be that largest number of points that can be shattered by the members of that class $f(x, \alpha)$ (Hastie, Tibshirani and Friedman, 2001). An illustration of a simple VC example given in Figure 3.3. As can be seen there are $2^3 = 8$ ways of assigning 3 points to two classes. For the displayed points in $\mathbb{R}^2$, all 8 possibilities can be realized using the separating hyperplanes. Thus the function class is said to shatter 3 points. This would not work if we were given 4 points, no matter how we placed them. See Figure 3.4 for an illustration for such a configuration. It is obvious from the illustration that this function class can shatter the white circles from the black dots. Therefore, the VC dimension of the class in Figure 3.3 is 3.



Suppose if $h < m$ is the VC dimension of the class of functions that a learning machine can implement, then for all of that class, the bound would be:

$$R[f] \leq R_{emp}[f] + \sqrt{\frac{1}{m} \left(h \left(\ln \frac{2m}{h} + 1 \right) + \frac{4}{\delta} \right)} \quad (3.13)$$

Assuming that the function class is independent of the underlying distribution of fixed unknown probability generating the data, with a probability of at least $1 - \delta$ over the drawing of the training samples. The Structural Risk Minimization approach tries minimize the right side of the bound rather than the empirical risk.

3.1.3 OPTIMAL SEPARATING HYPERPLANE

Our initial goal was to choose a classifier that will generalize the unseen examples well. Considering the Figure 3.5, there are many possible linear classifiers that can separate the data. The goal of optimal separating hyperplanes is to find a linear classifier that maximizes the margin, that is maximizes the distance between it and the nearest data point of each class. The expectation is that the optimal separating hyperplane well as opposed to other possible boundaries.

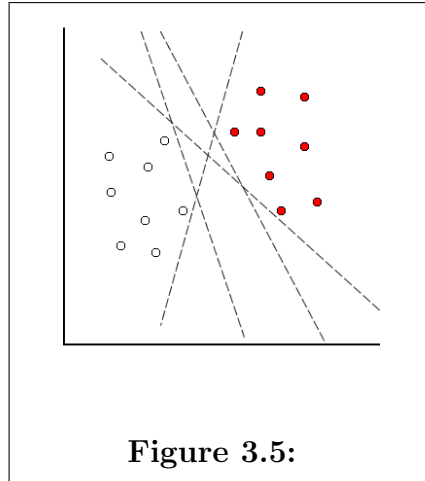


Figure 3.5:

To present the theoretical framework of optimal separating hyperplanes we will consider our original problem of separating a set of training observation belonging to two separate classes,

$$(x_1, y_1, \dots, (x_m, y_m) \in X \times \pm 1 \quad (3.14)$$

with a hyperplane defined in some dot product space $\mathcal{H}$,

$$\langle \mathbf{x}, \mathbf{x}' \rangle + b = 0 \text{ where } \mathbf{w} \in \mathcal{H}, b \in \mathbb{R} \quad (3.15)$$

corresponding to decision functions,

$$f(x) = \text{sgn}(\langle \mathbf{x}, \mathbf{x}' \rangle + b) \quad (3.16)$$

From all hyperplanes separating the data, there exists a unique optimal hyperplane, distinguished by the maximum of separation between any training point and hyperplane. To find such a plane we need to solve the following,

$$\max_{\mathbf{w} \in \mathcal{H}, b \in \mathbb{R}} \min \|\mathbf{x} - \mathbf{x}_i\| \Big| x \in \mathcal{H}, \langle \mathbf{x}, \mathbf{x}' \rangle + b = 0, i = 1, \dots, m \quad (3.17)$$

To find largest margin in the hyper planes we need to find the normal vector that leads to this margin. To construct a hyperplane with such a margin we solve,

$$\min_{\mathbf{w} \in \mathcal{H}, b \in \mathbb{R}} \tau(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2 \quad (3.18)$$

$$\text{subject to } y_i(\langle \mathbf{x}, \mathbf{x}' \rangle + b) \geq 1 \text{ for all } i = 1, \dots, m \quad (3.19)$$

The constraint ensures that $f(\mathbf{x}_i)$ will be $+1$ for $y_i = +1$ and -1 for $y_i = -1$. We need to minimize the length of $\mathbf{w}$ in Equation 3.18 because if $\|\mathbf{w}\| = 1$, the left hand side of Equation 3.19 would equal the distance from $\mathbf{x}_i$ to the hyperplane. Generally we have to divide $y_i(\langle \mathbf{x}, \mathbf{x}' \rangle + b)$ by the $\|\mathbf{w}\|$ to transform it into this distance. This will satisfy the restriction placed by Equation 3.19 and the overall margin will be maximized.

In Equation 3.18 the function τ is called the objective function, and the constraint in Equation 3.19 is referred to as the inequality constraints. These two equations combine to form the constrained optimization problem. Problems of this kind are dealt with by using Lagrange multipliers $\alpha_i \geq 0$

and a Lagrangian,

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^m \alpha_i (y_i (\langle \mathbf{x}, \mathbf{x}'_i \rangle + b) - 1) \quad (3.20)$$

The Lagrangian $\mathcal{L}$ has to be minimized with respect to the primal variables $\mathbf{w}$ and b and maximized with respect to the dual variables α_i . Formally,

$$\frac{\partial}{\partial b} \mathcal{L}(\mathbf{w}, b, \alpha) = 0 \text{ and } \frac{\partial}{\partial \mathbf{w}} \mathcal{L}(\mathbf{w}, b, \alpha) = 0 \quad (3.21)$$

leads to,

$$\sum_{i=1}^m \alpha_i y_i = 0 \quad (3.22)$$

and

$$\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i \quad (3.23)$$

By substituting Equation 3.22 and Equation 3.23 into the Lagrangian we can eliminate the primal variables $\mathbf{w}$ and b , arriving at the dual optimization problem. This problem solves,

$$\max_{\alpha \in \mathbb{R}^m} W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \quad (3.24)$$

$$\text{subject to } \alpha_i \geq 0 \text{ for all } i = 1, \dots, m \text{ and } \sum_{i=1}^m \alpha_i y_i = 0 \quad (3.25)$$

Using Equation 3.23, the hyperplane decision function in Equation 3.16 now can be re-written as,

$$f(x) = \text{sgn} \left(\sum_{i=1}^m y_i \alpha_i \langle \mathbf{x}_i, \mathbf{x}_j \rangle + b \right) \quad (3.26)$$

3.1.4 SUPPORT VECTOR CLASSIFICATION

The equation 3.26 completes all the tools necessary to describe the SVM. Since all the equation developed have been in the dot product space, $\mathcal{H}$. To express formulas in input patters in the domain X , we use the following mapping to,

$$k(x, x') := \langle \mathbf{x}, \mathbf{x}' \rangle = \langle \Phi(\mathbf{x}) \Phi(\mathbf{x}') \rangle \quad (3.27)$$

This enables us to express the vectors as dot products in terms of the kernel evaluated on the input patterns $\mathbf{x}, \mathbf{x}'$,

$$k(x, x') = \langle \mathbf{x}, \mathbf{x}' \rangle \quad (3.28)$$

This sort of substitution is formally called the kernel trick. This trick can be applied to all vectors in the feature space. Hence Equation 3.28 becomes,

$$f(x) = \text{sgn} \left(\sum_{i=1}^m y_i \alpha_i \langle \Phi(\mathbf{x}) \Phi(\mathbf{x}') \rangle + b \right) = \text{sgn} \left(\sum_{i=1}^m y_i \alpha_i k(x, x') + b \right) \quad (3.29)$$

and the following quadratic program form the dual problem becomes,

$$\max_{\alpha \in \mathbb{R}} W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j k(x_i, x_j) \quad (3.30)$$

$$\text{subject to } \alpha_i \geq 0 \text{ for all } i = 1, \dots, m \text{ and } \sum_{i=1}^m \alpha_i y_i = 0 \quad (3.31)$$

This is also referred to as the Lagrange dual function.

Theoretically all seems to fit, but in practice a separating hyperplane might not exist if there is a large overlap between the two classes. To allow for this overlap of observations slack variables are introduced, $\xi_i \geq 0$ for all $i = 1, \dots, m$, in order to relax the constraints of Equation 3.25 to,

$$y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i \text{ for all } i = 1, \dots, m \quad (3.32)$$

To find a classifier that generalizes well, both the classifier capacity, $\|\mathbf{w}\|$, and the sum of slacks $\sum_i \xi_i$ need to be controlled. The slacks provide an upper bound on the number of training errors.

It is possible to find a soft margin classifier that would incorporate the overlap of observation described above by minimizing the following objective function,

$$\tau(\mathbf{w}, \xi) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \quad (3.33)$$

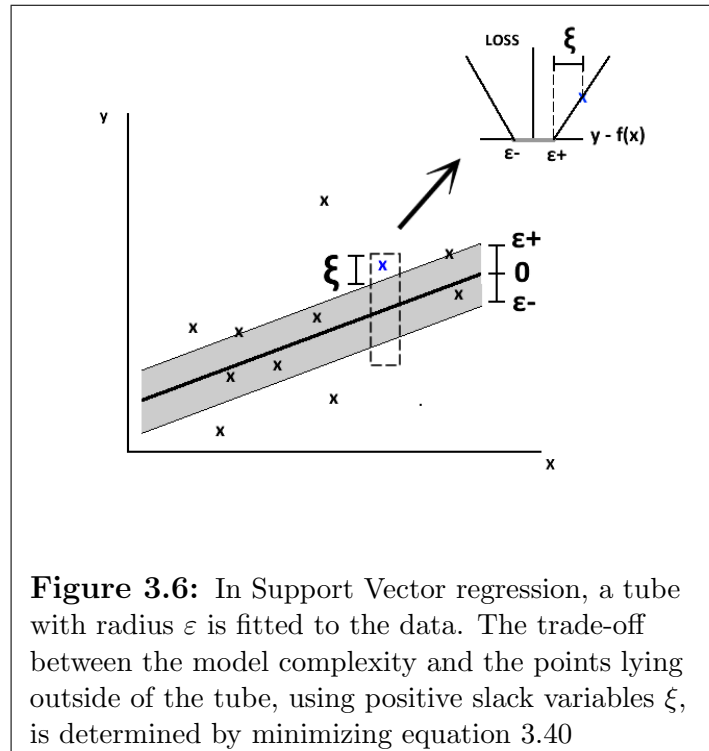
Subject to constraints,

$$\xi_i \geq 0 \text{ for all } i = 1, \dots, m \quad (3.34)$$

$$y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i \text{ for all } i = 1, \dots, m \quad (3.35)$$

The constant $C > 0$ determines the tradeoff between margin maximization and the training error.

3.1.5 SUPPORT VECTOR REGRESSION



Support Vector regression deals with estimating real valued functions. To use the SV theory to the regression case, an analog of soft margin is constructed in the space of the target values $y \in \mathbb{R}$. This can be accomplished by using ε -insensitivity loss function (Figure 3.6). This function calculates the loss by predicating $f(x)$ instead of the y as follows,

$$c(x, y, f(x)) = |y - f(x)|_\varepsilon := \max\{0, |y - f(x)| - \varepsilon\} \quad (3.36)$$

To perform linear regression, $f(x) = \langle \mathbf{w}, \mathbf{b} \rangle + b$, we need to minimize,

$$\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m |y_i - f(x)|_\varepsilon \quad (3.37)$$

Changing this to a constraint optimization problem such as the case in soft margin, we introduce slack variables. Two types of slack variables are needed ξ^0 and ξ^1 defined as follows,

$$\xi^0 := f(x_i) - y_i > \varepsilon \quad (3.38)$$

$$\xi^1 := y_i - f(x_i) > \varepsilon \quad (3.39)$$

The optimization problem is denoted as,

$$\min_{w \in \mathcal{H}, \xi^0, \xi^1 \in \mathbb{R}, b \in \mathbb{R}} \tau(\mathbf{w}, \xi^0, \xi^1) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m (\xi^1 + \xi^0) \quad (3.40)$$

subject to,

$$f(\mathbf{x}_i) - y_i \leq \varepsilon + \xi^0 \quad (3.41)$$

$$y_i - f(\mathbf{x}_i) \leq \varepsilon + \xi^1 \quad (3.42)$$

$$\xi^1, \xi^2 \geq 0 \text{ for all } i = 1, \dots, m \quad (3.43)$$

Generalizing this to Kernel based regression we introduce Lagrange multipliers same as in the case of classification. The following optimization problem is generated assuming $C, \varepsilon \geq 0$ are chosen a priori as follows,

$$\max_{\alpha, \alpha^*} = -\varepsilon \sum_{i=1}^m (\alpha_i * + \alpha_i) + \sum_{i=1}^m (\alpha_i * + \alpha_i) y_i - \frac{1}{2} \sum_{i=1}^m (\alpha_i * + \alpha_i) (\alpha_j * + \alpha_j) k(x_i, x_j) \quad (3.44)$$

$$\text{subject to } 0 \leq \alpha_i, \alpha_i^* \leq C \text{ for all } i = 1, \dots, m \text{ and } \sum_{i=1}^m (\alpha_i - \alpha_i^*) = 0 \quad (3.45)$$

The regression estimate is denoted as follows,

$$f(x) = \sum_{i=1}^m (\alpha_i * + \alpha_i) k(x_i, x) + b \quad (3.46)$$

This algorithm can be modified such that does not need to be specified a priori. In order to accomplish that an upper bound is specified on the fraction of points allows to lie outside the tube shown in Figure 3.6, and the ε is computed automatically. This is done by using the primal objective function,

$$\frac{1}{2} \|\mathbf{w}\|^2 + C |y_i - f(x_i)|_\varepsilon \quad (3.47)$$

Here $\varepsilon \geq 0$ is treated as a parameter over which we minimize.

3.2 GENETIC ALGORITHMS

Genetic algorithms (GA) are inspired by the evolutionary processes of natural selection as suggested by Darwins 1858 book, “The Origin of Species”. According to theory of biological evolution, organisms evolve and become tailored to survive in their own niche (Eiben and Smith, 1998). Similarly genetic algorithms (GA) in computer science are design to mimic these environmental processes. As in biological ecosystems the environmental factors determine the biological adaptation of the organism within the ecosystem, the GAs generate specific environment conditions in which only the fittest solutions in their population can survive. In this section a brief summary of the basics of GA will be given such that the reader may be able to understand the discussions in the subsequent chapters.

In natural ecosystems, environmental pressures fuel the natural selection process of the survival of the fittest. Organisms that can adapt quickly enough to these environmental pressures survive while others die off. The adaptation in biological organisms is accomplished by mating of fitter organism in the community, and through random mutation of their genome. In time the fitter parents produce more offspring and therefore the population of a specific organism will conform to the biological fitness of the fittest individuals. This causes a rise in the fitness of the population, where the evolved animal adapts to its environment.

The Genetic algorithms act as the ecosystem. They contain a population of possible solutions. They set criteria for choosing which solution will be allowed to reproduce and which will be removed from the population pool. To this end, they create environmental pressures to evolve their population such that the population conforms to the fitness of the best solution. As their biological counterparts, GAs use a mutation function which allows for the mutation of new off springs in the hope that these new mutated offspring might generate better solutions than the ones in the population.

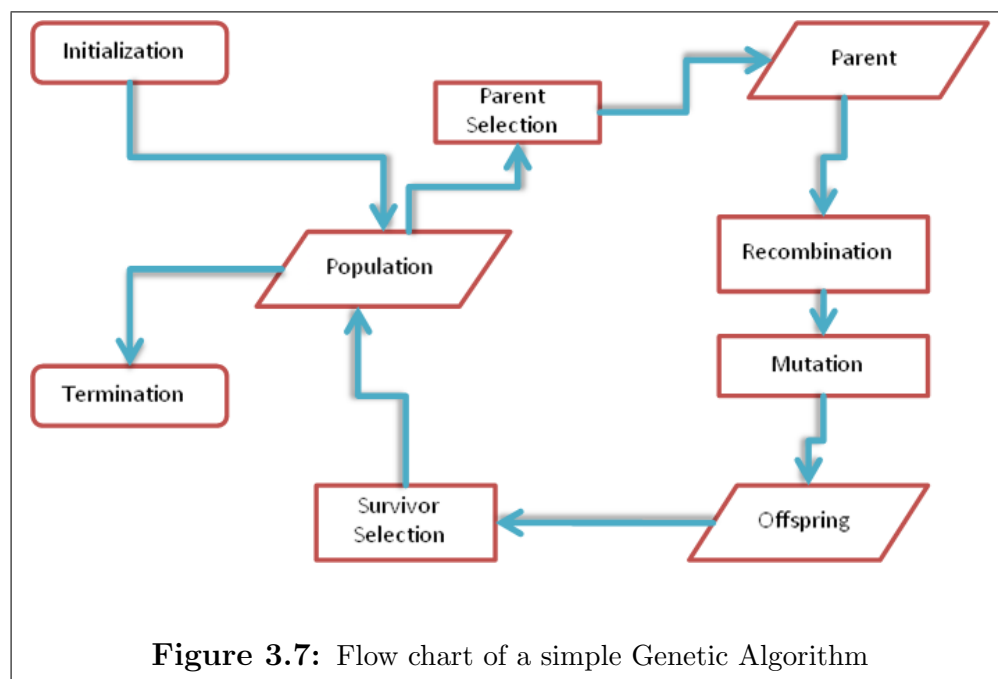


Figure 3.7: Flow chart of a simple Genetic Algorithm

Figure 3.7 shows a basic flow chart of a Genetic Algorithm. The main steps of a genetic algorithm and their purpose are as follows:

Initialization The initial population is randomly generated. To improve convergence times a more selective method might be employed to choose this initial population.

Population The population consists of member which are potential solutions to the search space. These members are represented in either binary, integer, floating point, or gray code form. The representation is called a chromosome, while a digit in the chromosome is called a gene. The genotype of the individual solution is the genes in the chromosome while the phenotype is the solution.

Parent Selection Parent selection is accomplished through many methods. Some of the most common methods used are fitness proportionate selection and tournament selection. In fitness proportionate selection the parents are categorized in terms of their phenotype. This categorization is implementation specific. From this categorization the fittest are randomly selected to mate and produce off springs. In tournament selection two parents are chosen at random and placed in a tournament. The fittest one according to a predetermined criteria is advanced into the matting pool of parents.

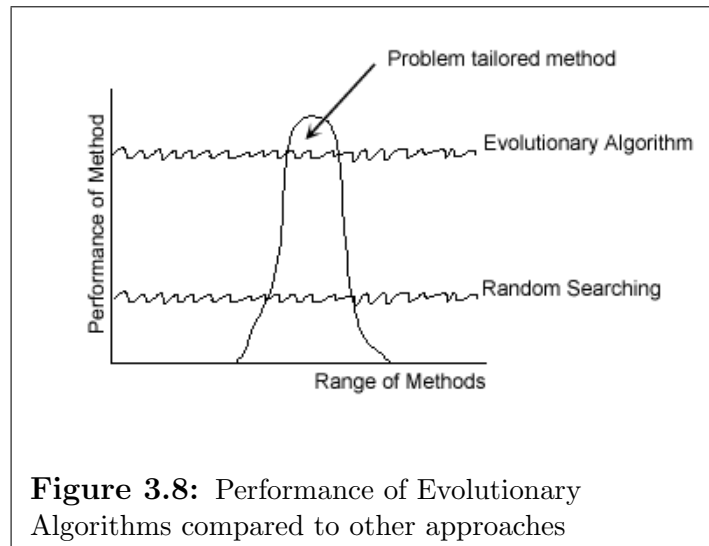
Recombination At this stage of the process the fittest parents are combines to produce new offspring. Different procedures are implemented and this stage of the process is a heavy area of research. The most simple technique is a one point crossover. In one point crossover the a random position is chosen, subject to the length of the chromosome. The genes of one parent are swapped into the chromosome of the other parent and vice versa. This part of the process allows the algorithm to search the local space for an optimal solution, formally called exploitation.

Mutation The new offspring are mutated using a predetermined algorithm custom fit to the problem being solved. The idea of mutation is to perform a random search of the search space and assuring the algorithm is not stuck in a local minima or maxima in the search space. This random search by the algorithm is called exploration of the search space.

Survivor Selection Non fit solutions are removed from the population at this stage. There are

different ways of accessing which solutions will be terminated. Usually a mixture of the average and low performers are terminated. The ideology behind this approach is that the population needs to maintain its diversity if an global optima is to be found. Through maintaining diversity the algorithm ensure that premature convergence on a local minima does not occur.

Termination The termination criteria of the algorithm is perhaps one of the most important. Termination condition most generally used is rate of partial improvement in the overall fitness of the population. This can mean either that a global optima has been reached or the algorithm is stuck in a local maxima. Other conditions specific to the problem being solved can also be applied.



GAs perform a heuristic based random search. They use historical information in their population of prospective solutions to direct the search in the direction of better overall fitness. The reproduction and mutation in GAs, allows them to be classified as random heuristic search algorithms. The evolution through reproduction and removing bad solutions allows the GAs to perform local searches within a niche of the search space.

The Genetic Algorithms are successful because they are able to balance the two form of searching: Exploration and Exploitation. Exploration is accomplished through the initialization with random vectors of the population and also through the mutation operators. For a given minimization function, a number of randomly generated set of candidate solutions, i.e. elements of the functions

domain, and apply the quality function as an abstract fitness measure, the higher the better (Eiben and Smith, 1998).

Figure 3.8 shows different approaches to solve read world problems. The graph shows that the performance of the problem tailored solution such as Support Vector Machines outperforms genetic algorithms but only to a selected few problems. These tailored solutions are typical based on some certain assumptions about the problem space and therefore cannot be implemented onto different scenarios. Evolutionary system which are an intelligent random search are able to address a wider range of problems and perform equally as well on all. They are considerably better than random searching as they employ exploitation as part of their methodology not just exploration, therefore they are classed as heuristic based random search algorithms.

THE SYSTEM DEVELOPED: SVM HISTORICAL SIMULATION OPTIMIZER

The system, SVM Historical System Optimizer, developed for experimenting with support vector machines on historical simulation. The design of the system is modular, because each of the modules was designed and tested separately for speed and accuracy. Later in the study, they were put together once a formal set of experiments had been developed. Setting a real world case scenario to test the experiments was essential to this study, therefore, at the heart of the system two important programs are used: FinCad and LIBSVM (Lin and Chang, 2001). These programs have been proven to provide the best performance, therefore providing a sound basis for experimentation. The data that was used in this study is real market data used by Calypso Technology for demonstration purposes. The section will briefly explain the system and the motivation behind each component.

The experimental system developed in this study was written in the Java programming language. This allowed for shorter development time as the task of creating this system was extensive. Any implementation of the Historical Simulation needed to be tested with the Calypso Risk Management Software before it could be used for experimentation purposes. This was necessary as the research was performed with idea of this methodology being merged into their risk software. Consequently the pricing environment and the historical simulation algorithm were the first to be implemented and tested before any intelligent based approach was devised.

The FinCad financial instrument pricing environment was incorporated into the optimizer to provide real market evaluations of scenarios. This allowed the system to generate real market quotes

for the non-linear financial instrument. The output of these market quotes were tested against the Calypso pricing environment as well as the results generated from the Historical Simulation for accuracy before any the incorporation of the LIBSVM library.

The LIBSVM library¹, the second system to incorporated in to the System. This library has been proven to generate respectable results. It holds great merit in the research community. Key institutions which have developed projects on this library include University of Chicago and University of Florida. This system has also won many awards one of which is the EUNITE worldwide competition on electricity load prediction in December 2001. The system provided testing of various kernels such as the linear, polynomial, radial basis function, and sigmoidal. A variety of options for support vector machines was also available: C -SVM, ν -SVM, ν -SVC, and ϵ -SVR.

Figure 4.1 shows a flowchart of the system. The key processes in the red boxes are the central components of generating the pricing environment for the financial instrument calculator as well as the components for calculating the Value-at-Risk using Historical Simulation. The processes labeled in green are vital to the Support Vector Machine algorithm, while the orange ones are important for the Historical Simulation algorithm.

The system requires two main files to generate the VaR measure from the Historical Simulation and the optimized Historical Simulation via SVM. The first file holds a portfolio of trades. Since the system only prices one type of non-linear instrument, Bermudan Swaption, therefore this file contains different parameters defining the Swaption. A list of these parameters is given in Table ???. The other file holds the option for the LIBSVM library and defines the type of SVM algorithm to be applied for the optimization of the Historical Simulation method. A list of these option are given in Table ??. Using these two files the system generates the VaR measures for each trade in the portfolio. It gives both the 99% Confidence VaR and the 99% Confidence VaR for the Historical Simulation and the Optimized Historical Simulation.

¹This library has been developed by two researchers Chih-Chung Chan and Chih-Jen Lin, and the software can be downloaded from the website, <http://www.csie.ntu.edu.tw/~cjlin/libsvm>.

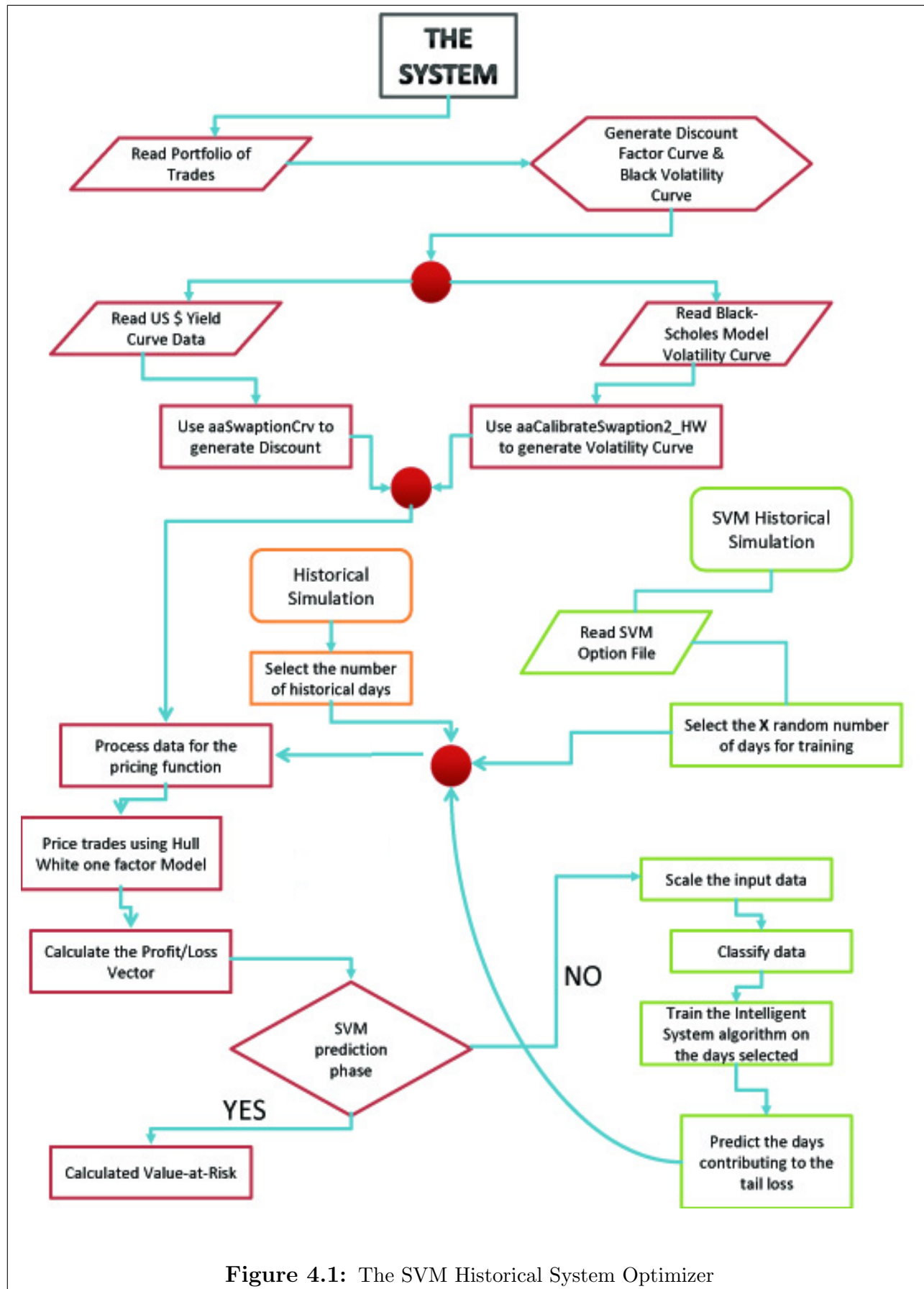


Figure 4.1: The SVM Historical System Optimizer

Option	Description
-s	Type of SVM (options C -SVM, ν -SVM, ν -SVC, and ϵ -SVR)
-t	Kernel Type (options linear, polynomial, radial basis, sigmoidal, pre-computed)
-g	γ of the Kernel Function
-n	The parameter ν of the ν -SVM
-e	Tolerance of the termination criterion

Table 4.1: Parameters of the SVM input file

Input	Description
d_v	Value Settlement Date
d_e	Effective date of the underlying swap.
d_m	Maturity Date of underlying swap
princ	Notional Principal Amount
cpn	Coupon Rate
freq	Payment frequency
acc	Accrual Method
d_{rul}	Business day convention
swpn	Swaption Type (payer - receiver)
d_{call}	First date on which the Bermudan option can be exercised
$d_{call-exp}$	Last date on which the Bermudan can be exercised
hl	Holiday list
df_{crv}	Discount factor curve
interp	Interpolation method
a	Mean revision constant
vlt-rate	Calculated from the Calibration Function
iter	number of time steps per coupon

Table 4.2: Parameter of the Bermudan Swaption

4.0.1 THE NON-LINEAR PRICING MODEL: BERMUDAN SWAPTION

The pricing model of choice used in this study was the Bermudan Swaption. The choice of the algorithm depended on the complexity of the models that are required in pricing. Bermudan are generally priced using a trinomial interest rate trees. Pricing a Bermudan is beyond the scope of this thesis. Hence a summary of the pricing model, the software used, and a basic explanation of a Swaption will be given.

4.0.1.1 EXAMPLE OF A SWAPTION: CASE EUROPEAN SWAPTION

Swaption are an Over-the-Counter available option to enter into a swap either as a fixed rate payer or a floating rate receiver. A Swaption has an underlying swap that will have an agreed maturity date, payment frequency and notational principle. At expiration, the buyer of the Swaption may either enter into a swap or the parties can agree to settle in cash. The swap rate is the strike interest rate, $i\%$ of the notational amount on the swap. An illustrated example of a Swaption follows:

Let us suppose that there is a UK corporation, ABC, that needs to borrow 100,000 from a corresponding bank, London Bank, in 2 years time for a further 3 years. The London Bank is currently lending at 100,000 at a 10% floating interest rate.

Lets assume that ABC believes Interest rates will rise over the next 2 years , thus putting them at risk of paying more interest rate than the current 10%.

In order to hedge the interest rate risk in the future, the ABC corporation will consider to buy a swap. The ABC corporation will buy a swap to exchange its floating rate payments for a fixed rate payment option, thus transforming the higher floating interest rate in two years time to a lower fixed interest rate loan. In order to carry out such an operation, ABC will need a 100,000 swap to pay fixed and receive floating beginning in 2 years time and an agreement that the swap will last for a further 3 years with a set frequency of payments, in this case let us assume annual payments.

If the floating interest rate drops from 10% to 9% the ABC corporation will still have to honor the swap and pay fixed 10% interest while receiving 9%, hence making a loss of 1% of the principle amount of the swap. To additionally hedge for such a possibility the ABC corporation needs to

buy an option on a swap, intuitively called a Swaption. Thus the ABC corporation can hedge this position by buying a 2-year European payer Swaption on a 3year pay fixed, receive floating swap at strike rate of 10%. If the swap rate in 2 years time is higher than 10% then the corporation can exercise the option.

However if the swap rates as suggested in situation B are 9%, then ABC can let the payer Swaption expire worthless and instead take out a swap at the current low swap rate of 9% to hedge its then position. As can be induced from the above arguments, the Swaption places a ceiling on the maximum fixed rate payable on the swap.

From this argument it is evident that this is a non-linear financial instrument. The difference between a European option and Bermudan is the exercise dates. The European Swaption has one underlying exercise date, while the Bermudan can be exercised on a set finite number.

4.0.2 SWAPTION PRICING MODEL: HULL WHITE SINGLE FACTOR MODEL

The model used to price the Bermudan Swaption was a Hull White Single Factor Model. The pricing of this model is very complex in nature and beyond the scope of this study. It is to be noted that the pricing model for the experiment is to be a black box, therefore as little knowledge as possible should be needed by the SVM optimize the Historical Simulation.

The mathematical form² is given as,

$$dr = [\theta(t) - \alpha r]dt + \sigma * dW \quad (4.1)$$

where the mean reversion and volatility parameters, α and σ are constant. The calculation of this model is performed by generating a tree structure for each exercise date. Before this function can be used it needs to be calibrated to market volatilities. The calibration function used by FinCad to adjust the Bermudan Swaption model to market volatilities attempts to minimize χ^2 error statistic on black volatility curves. The function itself is as follows,

²Adapted from FinCad Technical Documents for Developers

$$\chi^2 = \sum_{i=1}^m \left(\frac{BV_i - BV(state_i; parameters)}{\delta BV_i} \right)^2 = \left(\frac{\text{model error}}{\text{data uncertainty}} \right) \quad (4.2)$$

BV_i is the market-quoted Black volatility of the i^{th} calibration instrument. δBV_i is the estimated uncertainty in BV_i , for example the standard deviation of a sample of market quotes for the i^{th} calibrated instrument. N is the total number of calibration instruments and the $BV(state_i; parameters)$ is the model-predicted Black volatility of the i^{th} calibration instrument, where the $state_i$ is the evolution of the state for the i^{th} calibration instrument given trial values of the model parameters. The functions, BV_i and δBV_i are converted to price before calculating the χ^2 . The fitting algorithm varies the model parameters such that the minimum in the χ^2 error statistic is found.

EXPERIMENT: INTELLIGENT SYSTEM OPTIMIZATION OF HISTORICAL SIMULATION

Four experiments were designed for this study, but only three were finally implemented. The first experiment with Genetic Algorithms (GA) was never tested due to lack of appropriate implementation methodology. A brief summary of the proposed implementation of the GAs will be given in this section. Eventhough GAs were not directly used for optimization, but instead they were used to optimize the kernel and SVM parameters. The other three experiments performed use Support Vector Machines. The SVM were implemented in the endeavor to learn more about the problem area and to help explore novel techniques for optimizing Historical Simulation(HS). The experiments will be discussed in the order of their conception. The analysis section in this chapter summarizes the motivations and achievement of the experiments. The objective of the dissertation is stated again, which is to optimize historical simulation method used by major banks, and the Calypso Enterprize Risk Management Software (ERS).

Historical Simulation performed by the Calypso ERS software uses 500 days of historical data to calculated historical simulation. All the 500 days of historical data are fully priced and a profit and loss vector is generated. From this vector Value-at-Risk measure is calculated using the method described in the theoretical section. This puts a limit on the number of days that can be used by the Intelligent System, and also the places a benchmark of performance. Unfortunately due to the shortage of time none of the methodologies have been tested in the ERS software. A full pricing engine for Bermudan Swaptions is used by the system, SVM HS Optimizer, developed for

this study. Therefore all the results will be introduced in comparison to the time taken by the Historical Simulation. The number of days used to calculate the measure are also reported. The accuracy of the results are measured by running a full historical simulation on the full 500 days and generating VaR with both 95% confidence level, and 99% confidence level. The percent error, calculated as the percent difference between the full simulation and the SVM optimized simulation are presented in tabular format for each case of the experiments. Also a histogram representation is provided for a graphical depiction of which percentiles were populated by the SVM optimized Historical Simulation in order to calculate both the VaRs.

5.1 GENETIC ALGORITHMS

Extensive research was performed on Genetic Algorithms and their applications. The motivation behind using GAs is that they are very good general purpose optimizers. The objective was to of the study was to optimize historical simulation using an intelligent based approach, and that the implementation should not be problem specific.

In the wake of the objective a simple methodology was devised. Simple changes were suggested to the original GA algorithm. A 500 day limit was introduced for the number of full pricing. A sample of these 500 days would be taken to populate the GA environment. Each individual represented by a chromosome would be one historical market day, consisting of all the relevant values for that day. The fitness of the individual would be determined by the price generated from the market values represented by its chromosomes. The purpose of the algorithm would be to bring the total fitness of the population to low level, indicating that the population consisted of the lowest positions of the 500.

The implementation of the algorithm would have two extra features from the generic algorithm describe in the theoretical chapter: a) It would have a look-up table consisting of historical market data with knowledge of which days had been priced. b) It would also have a new offspring selection criteria which would compare the chromosome of the offspring with that of the historical market

days. If any of the days had similar values to that of the offspring the offspring would be assigned the value of that market day and added to the population.

The look-up table was designed to tackle the limit of 500 days of historical market data. The current historical simulation methods use approximately two years worth of market data to calculate Value-at-Risk(VaR). Therefore this was the maximum number of pricing the GA could perform. The new offspring selection criteria was also created in the same spirit.

The only problem with this implementation was a lack of stopping criteria for the algorithm. Although the limit of the 500 days existed, but the aim is to reduce the number of full pricings required to calculate the VaR measure. Unfortunately, the conventional stopping criteria such as percent improvement in the population cannot be used since new members of the population are only added if they are close to the values of a historical market day. There could be a situation where it would take a few generation before a new offspring was added to the population. Other parametric VaR could not be used either since that would limit the capability of getting decent results. Another problem that was envisaged was that the GAs are usually used to find global optima or minimums. Since the only finding the local maxima or minima is not the objective and there are a limited number of days to be chosen from, this methodology was not logically viable to be applied to the problem.

In light of these early difficulties the implementation of the GA was dropped in favor of the Support Vector Machines. Instead they were used to optimize a the ν -SVM optimization parameter and the γ for the radial basis kernel used in the SVM experiments. The parameters generated are given in Table 5.1. These parameter were used for the rest of the SVM experiments.

Parameters	Values
ν	0.03
γ	0.02
ϵ	1.0×10^{-13}

Table 5.1: Optimization Parameters for SVM

5.2 SUPPORT VECTOR REGRESSION ON PRICES

The first experiment conducted used a sample of 50 days from a total of 500 to train the Support Vector Regression (SVR) algorithm. The SVR was trained on the scaled prices of the Bermudan Swaption on those 50 days. The prices for the rest 450 days were then predicted using the SVR algorithm. The lowest 100 days as selected by the SVR were then fully priced. The first 50 days were added to these equalling 150 days of profit & loss positions to be used to calculate VaR. The VaR statistic was calculated assuming that the P&L consisted of 500 days worth of values, since the aim of the algorithm was to identify the values contributing to the tail loss. Therefore the tail in theory should have all the values that would be found in the tail of a fully priced Historical Simulation distribution. The VaR measures from this experiment consisting of one Bermudan Swaption are given below.

HS VAR	99% Confidence level	-457.26	95% Confidence Level	-344.044
SVM VAR	99% Confidence level	-290.243	95% Confidence Level	-132.506

As can be seen there was a significant difference between the SVM and HS calculations. The errors between the estimates are calculated as the percent difference between the Historical Simulation results and the SVM aided HS. There was a 36.5% error for the 99% confidence level, and a 61% error in the 95% confidence level VaR calculations. This error was quite large.

The histogram in Figure 5.1 of the P&L values generated by the two methods sheds some insight. As is apparent the values generated by the SVM aided historical simulation are not limited to the tail loss. They are uniformly distributed along the different P&L bins. This shows a deficiency in predicting the correct days that attribute to the tail loss by the SVM.

From this experiment it was concluded that approximating pricing models was not the most efficient way of optimizing Historical Simulation. Use of a new method was needed to get appropriate results.

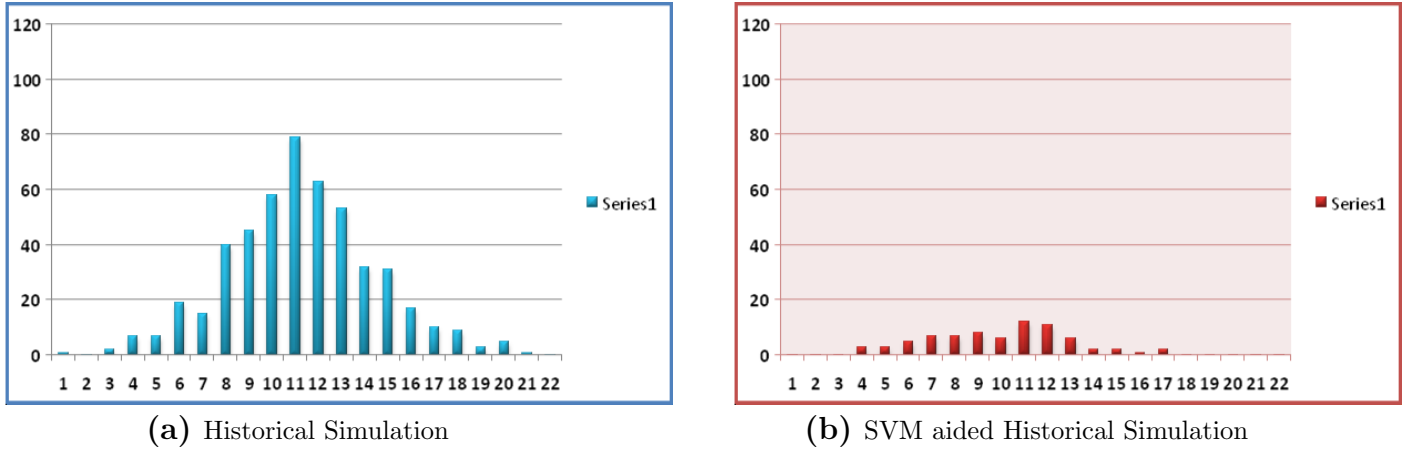


Figure 5.1: Case SVR-Pricing Approximations

5.3 SUPPORT VECTOR CLASSIFICATION ON P&L USING SIMPLE CLASSIFICATION

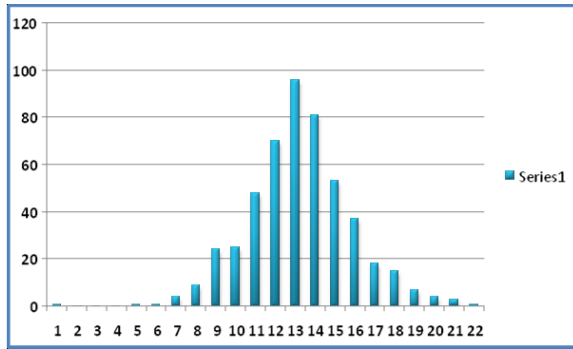
The new methodology developed to improve the SVM prediction. This new method aimed to predict negative P&L values instead of approximating the non-linear pricing function. Support Vector Classification algorithm was used to train on classified P&L values. Sample P&L values were generated from a sample of fully priced historical scenarios. Using this P&L, two cases were tested. In the first case the P&L was classified using a ± 1 hard limit. All the negative values were set to -1 while the positive were set to $+1$. In the second case, to differentiate between the extreme negative values a third class -2 was introduced. This class was generated by splitting the negative values of the sample P&L vector. The procedure used, was to first compute the mean of the negative values. The negative values above the mean were classified as -2 while the ones below the mean were classified as -1 . The experiments were run on a portfolio of 25 trades all consisting of Bermudan Swaptions. The results from each case was presented in terms of averages. Histogram of the full HS and the optimized HS are generated to provide insight into the effectiveness of this strategy.

5.3.1 CASE: OPTIMIZING HISTORICAL SIMULATION USING 2-CLASS CLASSIFICATION

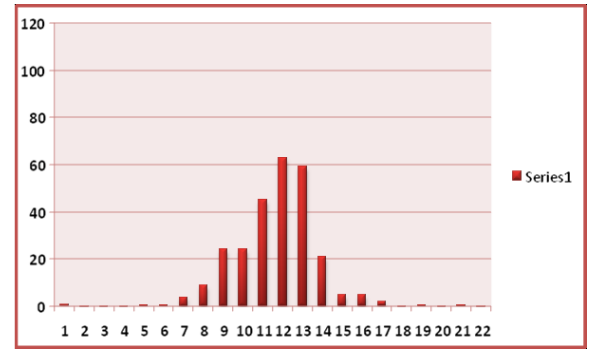
The 2-Class support vector classification method used a random sample of 50 training scenarios for each trade. The rest of the 450 scenarios of historical market data were classified and the negatives ones as predicted by the algorithm were then fully priced. The values of the first 50 were combined with the predicted negative values to generate the P&L vector. From this vector VaR was calculated. Table 5.2 summarizes the results from this experiment.

Training Days	Prediction Days	Total Days	Efficiency	Average Percent Error 99% VaR	Average Percent Error 95% VaR
50	290	340	32%	0%	0.12%

Table 5.2: Results of SVC 2-Class classification



(a) Historical Simulation



(b) SVM aided Historical Simulation

Figure 5.2: Case SVC Optimization using 2-Class classification

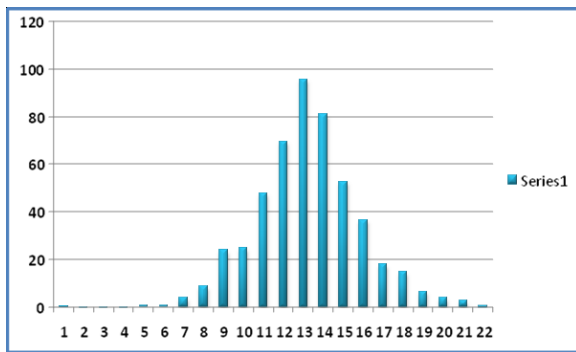
Eventhough this method used more than two hundred days to calculate the VaR measures, this new method performed far better in terms of accuracy than the method used in the first experiment. The histogram in Figure 5.2 illustrates the success of this method. The SVM aided HS negative tail histogram matches the HS negative tail. Both the histograms, HS and SVM aided HS, are the same up to the 11th P&L bin.

5.3.2 CASE: OPTIMIZING HISTORICAL SIMULATION USING 3-CLASS CLASSIFICATION

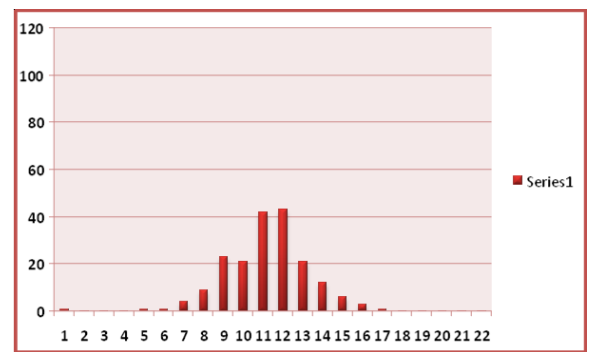
In this experiment the same method of training, and pricing were used. The introduction of the third class enabled for the sorting of the prediction made by the SVC algorithm. Therefore the 450 predicted historical days were sorted and the 175 negative ones were fully priced. Then the priced 225 days were used to generate the P&L vector through which VaR measure were calculated. This new method improved the efficiency of the SVM aided HS by 25%. This means that this algorithm would theoretically run in 45% of the time required to run a full Historical Simulation. Capping the number of days to 175 caused an increase in the percent error in the VaR. The amount of error is very small and is acceptable given the decrease in the amount of time taken to run the optimized algorithm. Table 5.3 summarizes the results obtained.

Training Days	Prediction Days	Total Days	Efficiency	Average Percent Error 99% VaR	Average Percent Error 95% VaR
50	175	225	55.25%	0.34%	2.2%

Table 5.3: Results of SVC 3-Class classification



(a) Historical Simulation



(b) SVM aided Historical Simulation

Figure 5.3: Case SVC Optimization using 3-Class classification

The histogram generated supports the assertion that only the historical market days contributing to the negative tail were chosen. The P&L bins at the peak of the distribution have been significantly

reduced. This indicates that less market days around the mean were priced. The negative tail of the optimized HS is intact, and matches the full Historical Simulation histogram. Therefore no information important to the calculation of VaR measures has been lost. The summary results in the Table 5.3 support this conclusion.

5.4 SUPPORT VECTOR REGRESSION ON P&L AND USING SIMPLE CLASSIFICATION

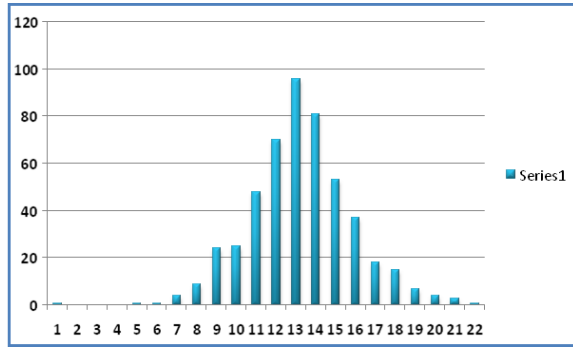
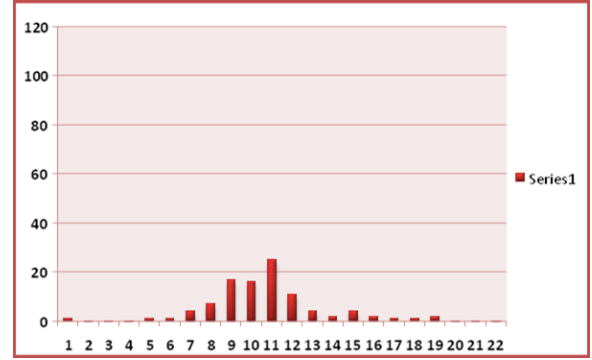
Support Vector Regression algorithm was employed to gain additive advantage of sorting the predicted values from the SVM algorithm. As the theoretic chapter asserts, the SVR algorithm was developed to harness the power of the SVMs for the real valued output domain. Three cases were used to test the advantage of using SVR for optimizing Historical Simulation. In the first two cases the same labeling mechanism was used as in the SVC algorithm experiments: 2-Class and 3-Class. The third experiment was carried out to test if there was any loss of information by applying a hard limit classification to the P&L vector. The number of training days and prediction days were reduced to 25 and 75 respectively. This was done on the basis of an experiment conducted to find the optimal days for accurate results. The summary of the finding are discussed in the Analysis section of this chapter.

5.4.1 CASE: OPTIMIZING HISTORICAL SIMULATION USING 2-CLASS CLASSIFICATION

Results from this case of the experiment are summarized in the Table 5.4. The efficiency of the optimized HS has increased from the best case of 55% when SVC where employed to 75%. This means that algorithm took approximately three quarters of less time to generate the same results, though the error on the VaR measures has increased. In the light of the efficiency gain the errors are acceptable for now.

The histogram analysis shows positive signs. It is important to note that the tail P&L bins

Training Days	Prediction Days	Total Days	Efficiency	Average Percent Error 99% VaR	Average Percent Error 95% VaR
25	75	100	75%	2.21%	8.19%

Table 5.4: Results of SVR 2-Class classification**(a)** Historical Simulation**(b)** SVM aided Historical Simulation**Figure 5.4:** Case SVR Optimization using simple 2-Class classification

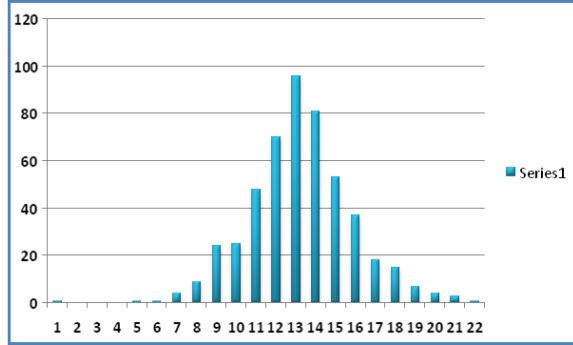
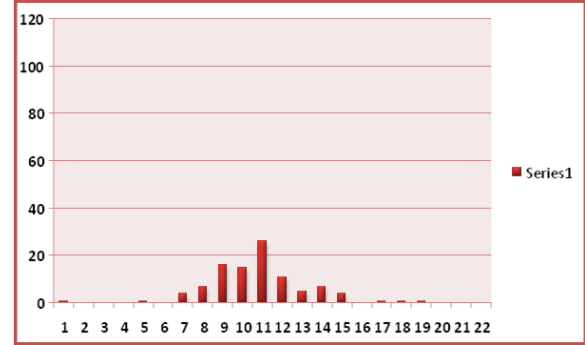
which are identical between the two histograms up to bin 8. This shows that the SVR algorithm is choosing the right historical market scenarios to price. The sorting capability is the most important gain from this methodology, which has brought down the total number of days priced.

5.4.2 CASE: OPTIMIZING HISTORICAL SIMULATION USING 3-CLASS CLASSIFICATION

After the attaining successful results from training SVR algorithm on ± 1 labels, it was necessary to test it on the 3 class case as well. The hope was that three classes would provide more information for the algorithm to train on. The results from this experiment are summarized in Table 5.5. Since the same number of training days and predicted days were priced as the 2-class case therefore there was no improvement in the efficiency of the method. The visible difference between providing extra information for the SVR was the improvement in the VaR measures. This was expected as there was more information for the support vector machine to learn from.

The histogram in Figure 5.4 supports the results presented. The optimized HS histogram of

Training Days	Prediction Days	Total Days	Efficiency	Average Percent Error 99% VaR	Average Percent Error 95% VaR
25	75	100	75%	1.1%	5.7%

Table 5.5: Results of SVR 3-Class classification**(a)** Historical Simulation**(b)** SVM aided Historical Simulation**Figure 5.5:** Case SVR Optimization using simple 3-Class classification

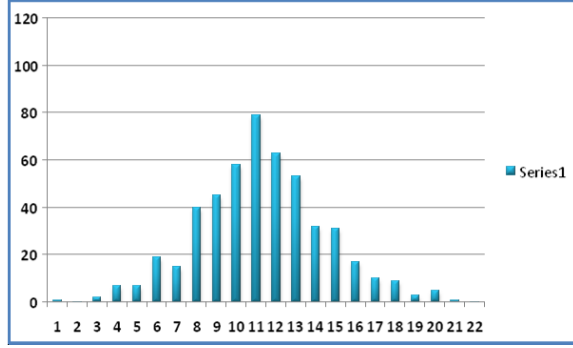
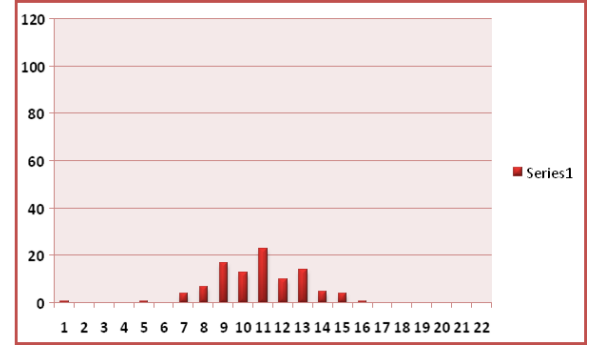
3-Class in comparison to the 2-Class one in Figure 5.6, it can be seen that there are less values in the P&L bins of the positive tail. The bin 16 is an obvious example.

5.4.3 CASE: OPTIMIZING HISTORICAL SIMULATION USING P&L

The motivation behind this experiment was to check the possibility of loss of information when simple classification is applied to the P&L of the sample days. In this experiment support vector regression was used to train on the scaled values of the P&L values directly. The results from this experiment are summarized in Table 5.6. Surprisingly the results from this method are close to the 2-class case. It was hypothesized that information was being lost by applying simple classification to the P&L distribution. The results disprove this idea. It seems that no added benefit was achieved by using the P&L results.

The histogram comparison of the optimized HS of this method is given in Figure 5.6. The histogram illustrates the what the results indicate. More values in the mean have been populated.

Training Days	Prediction Days	Total Days	Efficiency	Average Percent Error 99% VaR	Average Percent Error 95% VaR
50	75	100	75%	2.10%	8.2%

Table 5.6: Results of SVR using P&L**(a)** Historical Simulation**(b)** SVM aided Historical Simulation**Figure 5.6:** Case SVR Optimization using P&L

The same phenomenon was not observed in this case as compared to the optimized histograms in the 2-class and 3-class case, where only the negative tail was being populated.

5.5 ANALYSIS OF SVM EXPERIMENTS

The experiments conducted in this study showed that the most effective way of optimizing Historical Simulation method of calculative Value-at-Risk is through Support Vector Regression on 3-Class classified historical market scenarios. It was also learnt that approximating the pricing function is not the efficient way of approaching the problem, and training on the P&Ls was not effective either. Simplifying the problem domain by use of simple classification techniques proved to be the most effective method. Other experiments conducted which were not presented show the learning capability of the SVM, and the optimal number of days for training the SVR and the number of days selected to be priced via the algorithm such that more accurate VaR measures are obtained with the highest optimization. The analysis of the results obtained from all these experiments are summarized and discussed in this section.

The first experiment which employed Support Vector Regression on prices generated from the historical market rates produced very poor results. The reason attributed to the poor performance of support vector machines on instrument values and P&L was attributed to the dimensionality of the problem being learnt. The sample size were too small and the pricing function was very complex. Therefore, the function model developed by the SVM algorithm might not have been powerful enough to capture the complexity of the relationships in the data from the sample given. Due to this there was a decrease in the test performance of the Support Vector Machines.

After realizing the complexity issue, a new approach was attempted. Simple classification on the P&L results of the sample data was applied, which was test in the 2-Class case of the SVC. In this experiment the negative P&L values were classified as -1 and the positive values as $+1$. The results, attained with the reduction in the complexity of the output values, were far better than those in the first experiment. The SVC algorithm was able to more correctly indicate which days would produce a loss. The only issue with such an approach was that all the loss positions needed to be fully priced, since it could not be determined the amount of predicted loss generated from a given historical scenario.

The classification method was improved to indicate which historical scenarios would generate a greater loss. This was introduced in the experiment case of 3-class classification. The third class was generated by splitting the -1 labeled values into two classes. The criteria used for the division was through the mean of the P&L values. First the mean of all the loss values was calculated. Then any value lower than the mean was labels as -2 and any above the mean was labeled as -1 . In this manner a simple method was devised that would not add to the complexity of the problem being learned by the SVC. The addition of the third class allowed for the sorting of the predicted values by the SVC. The results indicated an increase in error. The error was negligible in comparison to the efficiency gain. In the 2-class experiment on average a total of 340 days were priced, while in the 3-class experiment 225 days were priced every time.

With a gain of 50% of efficiency in the case of the 3-Class SVC algorithm optimized HS, further experiments were developed to see if this could be improved. The motivation was to allow for a real valued output from the SVM such that it could be more effectively sorted. The use of Support

Vector Regression on hard limit label classified data is unconventional. No study was found that supported such a methodology. Since the difficulty in reducing the number of computations in the SVC experiments was attributed to hard limit output from the classifier, therefore employing support vector regression would allow for a possibility of sorting the outputs as the SVR is usually employed to deal with real outputs or targets. This would allow for better selection of historical market scenarios to be priced in order to generate the P&L vector for VaR calculation. Support for this methodology was found from the output of the first experiment, SVR on Prices, where the SVM was generating good average outputs for each of the Historical Market scenarios. If trained on a hard limit ± 1 label, it was hypothesized that it would conform to generate results close to zero for the data around the mean of the histogram, while larger real negative values would be given to negative outliers and larger real positive values for the positive outliers.

To test the theory the SVR algorithm was applied to the classified data using the same classification methodology as the SVC experiments. The efficiency gain was significant. For both the 2-Class and the 3-Class case the efficiency increased to 80%. This means that the optimized HS, was calculating VaR with little error using only 100 days of historical data as opposed to 500 days for the full HS.

Further experimentation was carried out to ensure that the results could not be enhanced. One key experiment which was included as a case of the SVR on P&L, was the training of the SVR algorithm directly on the scaled values of the sample data P&Ls. This experiment was carried out to see if there was any loss of information by classifying the data into the hard limit classes i.e. ± 1 and $-2 \& \pm 1$. The result showed that there was no significant loss of information as the error in the VaR measures was similar to that of the 2-class case. Also the histogram showed that the identification of the loss generating historical days was poor.

There was one other experiment carried out for completeness. This experiment was the basis of choosing 25 training days and 75 prediction evaluations to optimize HS for the SVR cases. In this experiment different sample sizes of data were chosen to train the SVR using 3-class classification, and Different sizes of prediction days were selected to be fully priced before the VaR measures were calculated. Based on the results obtained, which are summarized in Table 5.7, it was seen that the

#	Training Days	Prediction Days	Total Days	Efficiency	Average Percent Error 99% VaR	Average Percent Error 95% VaR
1	25	25	50	86.10%	4.76%	27.94%
2	50	25	75	81.24%	4.66%	20.71%
4	100	25	125	71.55%	4.79%	16.11%
5	25	50	75	81.33%	2.01%	7.77%
6	50	50	100	76.50%	2.73%	7.88%
7	100	50	150	66.73%	2.63%	10.15%
8	25	75	100	73.37%	1.15%	4.31%
9	50	75	125	68.75%	1.59%	5.01%
10	50	125	175	59.75%	0.59%	3.30%
11	75	125	200	55.19%	0.54%	2.48%
12	50	0	50	82.45%	37.70%	83.09%
13	100	0	100	73.39%	22.77%	51.60%
14	150	0	150	64.25%	16.62%	37.91%

Table 5.7: The effect of different training and prediction days chosen on VaR Measures.

best number of days for training the SVR were 25, while predicted days selected were 75. This combination produced the best VaR results with the highest efficiency.

It is important to note that there is a direct relationship between the number of training days chosen and the error in the VaR measures. The first three entries in Table 5.7, show this phenomenon. As the number of training days are increased the amount error also increases. This shows that there is over learning taking place by the algorithm which mounts to a decrease in the test performance. Choosing an optimal number of days is key for training is key such that over fitting does not take place.

The last three entries in the table have no prediction days. These cases were added to see if a random sample can generate similar results. The aim was to test the capabilities of the SVR algorithm. If the results generated by the random sample were close to those generated by the SVR then this would indicate that SVR had predictive capabilities classifying the historical market scenarios. Fortunately, the three random sample cases produce VaR measures with very large average error.

CONCLUSION

This dissertation is the result of a study performed for the Enterprize Risk System's(ERS) Team at Calypso Technology, London, UK. The problem it solved was raised to the team by prospective clients which was the amount of time required to run a full Historical Simulation for calculating the Value at Risk Measures. Due to the lack of research studies available in this problem domain, experiments were devised to gauge the effective method of optimizing the Historical Simulation. The methodology developed needed to reduce the time taken by the Historical Simulation without using any approximations which could affect the quality of the VaR measure generated. The solution also needed to be applicable to any instrument for which the VaR was being calculated.

Through course of the study two main Intelligent System techniques were chosen: Genetic Algorithms and Support Vector Machines. The first technique, Genetic Algorithms, were not used to optimize Historical Simulation due to concerns of their effective use as an optimizer. They were later used in the implementation of the SVMs to optimize certain parameters of the algorithm. To that end, they were successful. The second technique, SVM, were effectively used to optimize the Historical Simulation method of calculating VaR measures.

The performance gain in Historical Simulation(HS) through applying SVMs was approximately 74%. This meant that the optimized HS calculated the VaR measure in 26% of the time it took the full Historical Simulation to generate similar results. The number of days priced by the SVM in this case was 100 days as opposed to 500 days for the full HS. For more computationally intensive instruments the level of efficiency might raise to 80% as the time to train and run the SVM algorithm become negligible. This level of performance was far better than expected by the Calypso ERS team.

The contributions of this study to the problem domain include: a) The use of P&L distribution to train SVM in order to pick historical market scenarios that contribute to the tail loss in the distribution. b) The simple methodology of classifying P&L vectors which has proved efficient in reducing the complexity of the problem. c) The implementation of Support Vector Regression using hard limit classified labels which is unique as such an implementation has not been used before for optimization.

Some proposed improvements to this system include the incorporation of Genetic Algorithm to search for optimal parameters for the SVM algorithm. The use of differential genetic algorithms instead of generic genetic algorithms is a possible area of future research. It would be beneficial to find the difference in the quality of the SVM prediction when cross validation is used to approximate the SVM parameters, and when a genetic algorithm is applied. The trade off between the time taken to run the genetic algorithm with the level of accuracy attained would also be an interesting area of future research.

Other Intelligent System based techniques that can be applied in this problem domain include a new novel research called Regression by Feature Project (Guvenir and Uysal, 2006). This technique combines the simple K-nearest neighbor method with regression to predict better results. As showed in the 3-Class SVR case, by using a simple classification technique the predictions of the SVM increase by a considerable amount on the average. Therefore an algorithm built to take advantage of clustering and regression might prove to be more successful in this problem domain. It would also be interesting to see if Support Vector Regression could be implemented instead of the simple parametric regression to improve results.

REFERENCES

1. (2002). *Valuation and Risk Metrics*. Committee of Chief Risk Officers Valuation and Risk Metrics Working Group, 1 edition.
2. Artzner, P., Delbaen, F., Eber, J.-M., and Heath, D. (1998). Coherent measures of risk.
3. Barone-Adesi, G., Giannopoulos, K., and Vosper, L. (2000a). Backtesting derivative portfolios with fhs.
4. Barone-Adesi, G., Giannopoulos, K., and Vosper, L. (2000b). Filtering historical simulation. backtest analysis.
5. Basak, S. and Shapiro, A. (2001). Value-at-risk-based risk management: Optimal policies and asset prices. *The Review of Financial Studies*, 14(2):371–405.
6. Bekowitz, J. and O’Brien, J. (2002). How accurate are value-at-risk models at commercial banks. *The Journal of Finance*, 57(3):1093–1111.
7. Bishop, C. M. (2005). *Pattern recognition and Machine Learning*. Springer Science + Business Media Inc., draft 1.22 edition.
8. Christofferson, P., Hahn, J., and Inoue, A. (1999). Testing, comparing, and combining value-at-risk measure.
9. Cuthbertson, K. and Nitzsche, D. (2001). *Financial Engineering: Derivatives and Risk Management*. John Wiley & Sons Ltd., West Sussex, England.
10. Duffie, D. and Pan, J. (1997). An overview of value at risk.
11. Duffie, D. and Pan, J. (1999). Analytical value-at-risk with jumps and credit risk.
12. Eiben, A. E. and Smith, J. E. (2003). *Introduction to Evolutionary Computing*. Natural Computing Series. Springer-Verlag.
13. Engelbrecht, R. (2003). *A Comparison Of Value-At-Risk Methods For Portfolios Consisting Of Interest Rate Swaps And FRAs*. Ph.d., University of the Witwatersrand, Johannesburg.

14. Engle, R. F. and Manganelli, S. (1999). Caviar: Conditional autoregressive value at risk by regression quantiles. *University of California*.
15. Forrest, S. (1993). Genetic algorithms: Principles of natural selection applied to computation. *Science, New Series*, 261(5123):872–878.
16. Gençay, R. and Selçuk, F. (2004). Extreme value theory and value-at-risk: Relative performance in emerging markets. *Internal Journal of Forecasting*, 20:287–303.
17. Gençay, R., Ulugülayağci, A., and Selçuk, F. (2003). High volatility, thick tails and extreme value theory in value-at-risk estimation. *Insurance: Mathematics and Economics*, 33:337–356.
18. Gibson, M. S. and Pritsker, M. (2000). Improving grid-based methods for estimating value at risk of fixed-income portfolios.
19. Glasserman, P., Heidelberger, P., and Shahabuddin, P. (1999a). Importance sampling and stratification for value-at-risk. *Computational Finance*, pages 7–24.
20. Glasserman, P., Heidelberger, P., and Shahabuddin, P. (1999b). Importance sampling and stratification for value-at-risk.
21. Glasserman, P., Heidelberger, P., and Shahabuddin, P. (1999c). Stratification issues in estimating value-at-risk. *IEEE Computer Society Press*, pages 351–358.
22. Glasserman, P., Heidelberger, P., and Shahabuddin, P. (2000a). Efficient monte carlo methods for value-at-risk. *Columbia University*.
23. Glasserman, P., Heidelberger, P., and Shahabuddin, P. (2000b). Portfolio value-at-risk with heavy-tailed risk factors.
24. Glasserman, P., Heidelberger, P., and Shahabuddin, P. (2000c). Variance reduction techniques for estimating value-at-risk. *Management Science*.
- Harmantzis et al.. Harmantzis, D. F. C., Miao, L., and Chien, Y. Empirical study of value at risk and expected short fall models with heavy tails.
25. Hastie, T., Tibshirani, R., and Friedman, J. (2001). *The Elements of Statistical Learning: Data Mining, Inference and Prediction*. Springer Series in Statistics. Springer, New York, USA.
26. Holton, G. A. (2003). *Value-at-Risk: Theory and Practice*. Academic Press, London.
27. hsiung Lin, C. and Chien, C.-C. C. (2002). Incorporating a ged model into the historical simulation method for value-at-risk.
28. Jennison, C. and Sheehan, N. (1995). Theoretical and empirical properties of the genetic algorithm as a numerical optimizer. *Journal of Coputational and Graphical Statistics*, 4(4):296–318.

29. Jorion, P. (2002). *Value at Risk*. McGraw-Hill, London, 2 edition.
30. Laubsch, A. J. (1999). *Risk Management: A Practical Guide*. RiskMetrics Group, 1 edition.
- Li and Mascagni. Li, Y. and Mascagni, M. Grid-based monte carlo application. Master's thesis, Florida State University.
31. Lin, C.-J. and Chang, C.-C. (2001). Libsvm: Library for support vector machines.
32. Liu, Y. (2005). Value-at-risk model combination using artificial neural.
- Locarek-Junge and Prinzler. Locarek-Junge, H. and Prinzler, R. Estimating value-at-risk using neural network.
- Mangasarian and Musicant. Mangasarian, O. L. and Musicant, D. R. Lagrangian support vector machines.
33. Mina, J. (2000). Calculating var through quadratic approximations. *RiskMetrics*.
34. Mina, J. and Ulmer, A. (1999). Delta-gamma four ways. *RiskMetrics Group, LLC*.
35. Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill, Singapore, international edition.
36. Moosa, I. A. and Bollen, B. (2002). A benchmark for measuring bias in estimated daily value at risk. *International Review of Financial Analysis*, 11(1):85–100.
37. Oesterreicher, I., Mitschele, A., and Schlottmann, F. (2000). Comparison of multi-objective evolutionary algorithms in optimizing combinations of reinsurance contracts. *ACM*.
38. Price, K. V., Storn, R. M., and Lampinen, J. A. (2005). *Differential Computing*. Natural Computing Series. Springer-Verlag, Berlin, Heidelberg.
39. Prinzler, R. (1999). Reliability of neural network based value-at-risk estimates.
40. Pritsker, M. (2001). The hidden dangers of historical simulation.
41. Scholkopf, B. and Smola, A. J. (2002). *Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, Cambridge.
42. Vapnik, V. (2001). *The Nature of Statistical Learning Theory*. Springer-Verlag, New York.
43. Wei, G. C. G. and Tanner, M. A. (1990). A monte carlo implementation of the em algorithm and the poor man's data augmentation algorithms. *Journal of the American Statistical Association*, 85(411):699–704.
44. Wo, X. and Yanfeng (2005). A quantile-data mapping model for value-at-risk based on bp and support vector regression. *WINE, LNCS 3828*:1094–1102.
- Wong et al.. Wong, M. C. S., Cheng, W. Y., and Wong, C. Y. P. Market risk management of banks: Implications from the accuracy of value-at-risk forecasts.